Auditoría de SOLOServis

Qué encontramos en migraciones, esquema, flujos de escritura, pruebas y bases desechables

Proyecto SOLOServis

15 al 17 de octubre de 2026

Resumen

Acá juntamos en un solo lugar los seis pasos de la auditoría del día 1 sobre SOLOServis (rama `feature/services-cart-and-comparison-new`): (1) rama, estado local y migraciones, (2) esquema de productos, tiendas, ofertas e historial, (3) cómo se escriben productos y ofertas, (4) si el backend compila y qué pruebas hay, (5) un diagnóstico en bases desechables y (6) una revisión estática de Kubernetes y del despliegue. En el repositorio no tocamos nada: todo fue lectura. Lo único que ejecutamos fueron dos contenedores de usar y tirar, que ya borramos; el Paso 6 fue lectura pura, sin tocar ningún cluster.

Índice

1. Paso 1: rama, estado local y migraciones **4**

1.1. Comandos que corrimos . . . 4
1.2. Rama y estado local . . . 4
1.3. Últimos 5 commits . . . 4
1.4. Migraciones de este checkout (11 archivos) . . . 4
1.5. Ramas remotas (13) y fecha de su último commit . . . 5
1.6. Cómo está `origin/main` . . . 5
1.7. Migraciones por rama remota . . . 6
1.8. Lo más importante . . . 6
1.9. Para preguntarle al equipo . . . 6
1.10. Cómo quedó el Paso 1 . . . 7

2. Paso 2: el esquema de productos, tiendas, ofertas e historial **8**

2.1. Qué miramos y qué no . . . 8
2.2. Ficha de cada tabla . . . 8
2.2.1. `product_category` . . . 8
2.2.2. `product` . . . 8
2.2.3. `product_category_specification` . . . 9
2.2.4. `product_specification_value` . . . 10
2.2.5. `store` . . . 11

2.2.6. product_offer . . . 11
2.2.7. product_price_history . . . 12
2.3. Resumen en una tabla . . . 13
2.4. Riesgos y dudas . . . 14
2.5. Solo lectura y estado de Git . . . 15

3. Paso 3: cómo se escriben productos y ofertas 16
3.1. Rutas HTTP . . . 16
3.2. Consultas en backend/database/queries/products.sql . . . 16
3.3. Ficha de cada flujo de escritura . . . 17
3.3.1. Ingesta de productos . . . 17
3.3.2. Ingesta de ofertas . . . 18
3.3.3. Crear oferta (POST /offers) . . . 18
3.3.4. Actualizar oferta (PUT /offers/{id}) . . . 19
3.3.5. Crear producto (POST /products) . . . 19
3.3.6. Actualizar producto (PUT /products/{publicID}) . . . 20
3.3.7. Baja de producto (DELETE /products/{publicID}) . . . 20
3.3.8. Otros que escriben: tiendas y especificaciones . . . 21
3.4. Respuestas puntuales . . . 21
3.5. sqlc y pruebas . . . 22
3.6. Cuándo se genera historial . . . 22
3.7. Riesgos y dudas . . . 22
3.8. Solo lectura y estado de Git . . . 23

4. Paso 4: línea base de pruebas y compilación 24
4.1. Cerrando el Paso 3 . . . 24
4.1.1. Estado real de Git . . . 24
4.1.2. Archivos de prueba . . . 24
4.1.3. Código que parece inalcanzable en ingestion.go . . . 24
4.1.4. Chequeo por muestreo de líneas citadas en el Paso 3 . . . 25
4.1.5. Autenticación . . . 26
4.2. Chequeos antes de correr . . . 26
4.3. Línea base . . . 27
4.4. Fallos que ya existían . . . 27
4.5. Ojo con el alcance . . . 27
4.6. Dónde quedaron los logs . . . 27
4.7. Preguntas nuevas . . . 28
4.8. Cierre . . . 28

5. Paso 5: diagnóstico en bases desechables **29**

5.1. Antes de empezar . . . 29
5.2. Aplicando las migraciones . . . 29
5.3. Qué pasa con los prefijos repetidos . . . 29
5.4. Consultas de diagnóstico . . . 29
5.5. Revisando el esquema en A . . . 30
5.6. Archivos de trabajo . . . 31
5.7. Limpieza . . . 31
5.8. Hallazgos nuevos y preguntas . . . 31
5.9. Cierre . . . 32

6. Paso 6: Kubernetes y despliegue (revisión estática) **33**

6.1. Preparación . . . 33
6.2. Inventario . . . 33
6.2.1. Archivos de Kubernetes y script . . . 33
6.2.2. Carpeta `docker/` . . . 34
6.2.3. Recursos . . . 34
6.3. Migraciones en el cluster . . . 35
6.3.1. De dónde salen y en qué orden se aplican . . . 35
6.3.2. `schema_migrations`, reintentos y atomicidad . . . 35
6.3.3. Migraciones por rama . . . 36
6.3.4. ¿Se cargan seeds demo fuera de pruebas? . . . 36
6.4. PostgreSQL . . . 36
6.5. APIs, frontend y Dockerfiles . . . 37
6.6. Ingress y nginx . . . 38
6.7. El script `scripts/k8s/deploy.ps1` . . . 40
6.8. Seguridad y operación . . . 41
6.9. Pruebas y validación . . . 42
6.10. Contraste con los Pasos 1 a 5 . . . 42
6.11. Hallazgos nuevos y preguntas . . . 43
6.12. Cierre . . . 43

1. Paso 1: rama, estado local y migraciones

Fecha: 15-10-2026. La idea era ver en qué rama estamos y cómo están las migraciones, sin cambiar nada.

1.1. Comandos que corrimos

```
git status --short
git branch --show-current
git log -5 --oneline
Get-ChildItem .\backend\database\migrations -File | Sort-Object Name | Select-Object -
    ExpandProperty Name
git fetch --all --dry-run
git branch -r
git log origin/main --oneline -5
git for-each-ref --sort=-committerdate refs/remotes --format="%(committerdate:short) %(refname:
    short)"
git ls-tree --name-only <rama> backend/database/migrations/   (por cada rama remota)
```

1.2. Rama y estado local

Estamos en `feature/services-cart-and-comparison-new` y el árbol de trabajo está limpio: `git status -short` no devolvió nada.

1.3. Últimos 5 commits

```
2738771 Merge branch 'main' into feature/platform-apis-and-product-comparison
        (HEAD; coincide con origin/feature/services-cart-and-comparison-new,
        origin/feature/platform-apis-and-product-comparison y la rama local
        feature/platform-apis-and-product-comparison)
f103868 Merge pull request #23 from DoSafills/Feat-Services
f264f89 Merge pull request #20 from DoSafills/feature/store-add-to-cart
8b2ac32 Merge branch 'feature/services-cart-and-comparison-new' into Feat-Services
        (origin/Feat-Services, Feat-Services)
2990708 arreglo rapido
```

1.4. Migraciones de este checkout (11 archivos)

- `001_schema.sql`
- `002_views.sql`
- `003_seed.sql`
- `004_platform_apis.sql`
- `005_demo_catalog.sql`
- `006_demo_catalog_services.sql`
- `006_refrigeradores.sql`
- `007_Computacion.sql`
- `007_service_cart.sql`
- `008_normalize_prices_to_clp.sql`
- `008_Telefonia.sql`

Cosas que llaman la atención:

- Hay prefijos repetidos: dos archivos 006, dos 007 y dos 008.
- No existen los archivos 009 ni 010, aunque el README del backend sí los menciona.
- Los nombres no siguen un estilo único: `007_Computacion` y `008_Telefonia` empiezan con mayúscula y el resto va en minúscula con guión bajo.
- El orden de listado cambia según la herramienta. PowerShell (`Sort-Object Name`, sin distinguir mayúsculas) pone `008_normalize_prices_to_clp` antes que `008_Telefonia`; `git ls-tree` (orden por bytes, mayúsculas primero) las pone al revés. Los pares 006 y 007 salen igual en ambas.
- El riesgo es que, si las migraciones se aplican por orden alfabético (por ejemplo con `initdb.d` de Docker), el orden de los dos 008 varíe según el sistema. Falta leer los archivos (sin ejecutarlos) para ver si `008_normalize_prices_to_clp` depende de datos que crea `008_Telefonia`.

1.5. Ramas remotas (13) y fecha de su último commit

Fecha	Rama
2026-10-07	`origin/feature/platform-apis-and-product-comparison`
2026-10-07	`origin/feature/services-cart-and-comparison-new` (la actual)
2026-10-06	`origin/Feat-Services`
2026-10-06	`origin/feature/store-add-to-cart`
2026-10-06	`origin/feature/Store-comparison`
2026-10-06	`origin/feature/k8s-mvp-deploy`
2026-10-05	`origin/refactor/optimizacion-reutilizacion`
2026-10-02	`origin/develop`
2026-10-02	`origin/Feat/Carrito`
2026-09-25	`origin/Camilo-Interfaz`
2026-09-24	`origin/Camilo`
2026-09-10	`origin/main`

`origin/HEAD` apunta a `origin/main`; la línea `origin` que sale en la salida es esa referencia, no una rama de verdad.

1.6. Cómo está `origin/main`

```
a97bc75 Add files via upload
409f190 Delete docs/mer informe V1.1.pdf
2024c78 Documentos Sprint 0
5f9fef5 chore: initialize SoloServis with pnpm workflow
```

`main` solo tiene commits de inicialización y documentos del Sprint 0, ni siquiera tiene la carpeta `backend/database/migrations`. Sirve poco como referencia: está muy atrasada respecto de las ramas de trabajo.

Sobre `git fetch -all -dry-run`: no mostró líneas `From ...`, así que al parecer no hay commits remotos pendientes. Las líneas `Enumerating/Compressing/Writing objects` parecen mantenimiento automático de Git y no tocan archivos del proyecto.

1.7. Migraciones por rama remota

Buscamos 009 y 010 en todas las ramas y no hay ninguna. Lo que sí hay son cuatro líneas de migraciones distintas:

Línea A: "platform-apis" completa (11 archivos, hasta 008). Ramas: `feature/platform-apis-and-prod` `feature/services-cart-and-comparison-new` y `Feat-Services`. Archivos: 001_schema, 002_views, 003_seed, 004_platform_apis, 005_demo_catalog, 006_demo_catalog_services, 006_refrigeradores, 007_Computacion, 007_service_cart, 008_Telefonia, 008_normalize_prices_to_clp.

Línea B: "platform-apis" sin servicios ni carrito (8 archivos, hasta 008). Ramas: `feature/Store-compar` `feature/k8s-mvp-deploy` y `feature/store-add-to-cart`. Archivos: 001_schema, 002_views, 003_seed, 004_platform_apis, 005_demo_catalog, 006_refrigeradores, 007_Computacion, 008_Telefonia. Le faltan, respecto de A: `006_demo_catalog_services`, `007_service_cart` y `008_normalize_prices_t`

Línea C: "develop" (5 archivos, hasta 005). Ramas: `develop` y `refactor/optimizacion-reutilizacion`. Archivos: 001_schema, 002_views, 003_seed, 004_user_email_verified, 005_seed_catalogo. Trae `user_email_verified`, que no existe en A ni en B.

Línea D: "Camilo" (histórica). `origin/Camilo`: 001 a 003. `origin/Camilo-Interfaz`: 001 a 003 más 004_products, 005_electrodomesticos y 006_celulares. `origin/Feat/Carrito`: lo anterior más 007_services.

Dónde chocan los números (las líneas se separan después de la 003):

- 004: `platform_apis` (A/B) | `user_email_verified` (C) | `products` (D)
- 005: `demo_catalog` (A/B) | `seed_catalogo` (C) | `electrodomesticos` (D)
- 006: `demo_catalog_services` y `refrigeradores` (A) | `celulares` (D)
- 007: `Computacion` y `service_cart` (A) | `services` (D)

1.8. Lo más importante

1. La rama actual está limpia, sin cambios locales.
2. Está en el mismo commit que `feature/platform-apis-and-product-comparison`; ambas son las más recientes (7 de octubre) y tienen las mismas 11 migraciones.
3. Las migraciones 009 y 010 del README no existen en ninguna rama remota.
4. Hay 4 líneas de migraciones sin reconciliar y con números que se pisan.
5. A y B no incluyen `user_email_verified` (línea C). Hay que revisar en el código y el esquema si el backend de la línea A lo necesita; no hay que darlo por hecho.
6. Los prefijos repetidos (006, 007, 008) dejan el orden a merced de la herramienta.
7. `main` no sirve de referencia para migraciones.

1.9. Para preguntarle al equipo

1. ¿Cuál rama o línea es la referencia del proyecto (A, C u otra)?
2. ¿Las migraciones 009 y 010 del README existen en algún equipo local o rama sin publicar, o nunca se crearon?
3. `user_email_verified` (línea C): ¿se incorpora a A o se descartó? ¿Hay plan de reconciliar `develop` con `platform-apis`?
4. ¿Cómo se aplican hoy las migraciones: por orden alfabético en Docker (`initdb.d`) o con alguna herramienta? ¿Cómo se resuelve el orden de los archivos con prefijo repetido?

5. ¿Qué versión de migraciones está desplegada en cada entorno?

1.10. Cómo quedó el Paso 1

Rama y estado local registrados.

Lista de migraciones registrada.

Diferencia entre el checkout y el README documentada (faltan 009 y 010).

Ramas remotas y sus migraciones comparadas.

No se tocó código, migraciones ni datos.

Pendiente para el Paso 2: clasificar los SQL como DDL, seed o vistas (ya quedó hecho en el informe del Paso 2).

2. Paso 2: el esquema de productos, tiendas, ofertas e historial

Fecha: 16-10-2026. Lectura estática de los archivos SQL y las consultas.

2.1. Qué miramos y qué no

Revisamos `backend\database \migrations \001_schema.sql`, el resto de los SQL de esa carpeta y las consultas de `backend\database \queries \products .sql`. No ejecutamos SQL ni migraciones y no consultamos ninguna base. Esto describe lo que hay en este checkout; no confirma cómo está el esquema en otros entornos.

2.2. Ficha de cada tabla

2.2.1. `product_category`

Definida en `001_schema.sql:46`.

Columna	Tipo	Nulabilidad	Default
`id`	SERIAL	NOT NULL (PK)	—
`parent_category_id`	INTEGER	NULL	—
`name`	VARCHAR(150)	NOT NULL	—
`description`	TEXT	NULL	—
`active`	BOOLEAN	NOT NULL	TRUE
`created_at`	TIMESTAMP	NOT NULL	NOW()
`updated_at`	TIMESTAMP	NOT NULL	NOW()

Clave primaria
`id`

Claves foráneas
`parent_category_id` → `product_category.id`, ON DELETE SET NULL. ON UPDATE no se indica, así que PostgreSQL usa NO ACTION.

UNIQUE / CHECK
No hay.

Índices
La PK sobre `id`. El `idx_product_category` está en `product.category_id`, no acá.

Triggers
Ninguno.

Quién la toca después
`003_seed.sql:19,22`; `005_demo_catalog.sql:21`; `006_refrigeradores.sql:21`; `007_Computacion.sql:22,3` (inserts) y `007_Computacion.sql:28-33` (actualiza `parent_category_id`); `008_Telefonia.sql:22`.
No hay ALTER TABLE posterior.

Ojo con
`name` no es UNIQUE. Hay que decidir si se aceptan categorías con el mismo nombre y cómo se reconocen las que ya existen.

2.2.2. `product`

Definida en `001_schema.sql:56`.

Columna	Tipo	Nulabilidad	Default
id	SERIAL	NOT NULL (PK)	—
public_id	UUID	NOT NULL	gen_random_uuid()
category_id	INTEGER	NOT NULL	—
brand_id	INTEGER	NULL	—
name	VARCHAR(255)	NOT NULL	—
model	VARCHAR(150)	NULL	—
sku	VARCHAR(100)	NULL	—
description	TEXT	NULL	—
active	BOOLEAN	NOT NULL	TRUE
created_at	TIMESTAMP	NOT NULL	NOW()
updated_at	TIMESTAMP	NOT NULL	NOW()

Clave primaria
id

Claves foráneas
category_id → product_category.id, ON DELETE RESTRICT. brand_id → brand.id, ON DELETE SET NULL. ON UPDATE no se indica (NO ACTION).

UNIQUE
public_id y sku (el SKU admite NULL, o sea que no todo producto tiene que tener uno).

CHECK
No hay.

Índices
PK (id); UNIQUE en public_id y sku; idx_product_category (category_id); idx_product_brand (brand_id).

Triggers
Ninguno.

Quién la toca después
003_seed.sql:31; 005_demo_catalog.sql:87,193,207; 006_refrigeradores.sql:69,86,103; 007_Computacion.sql:84,101,118; 008_Telefonia.sql:65,82,99 (todos inserts). No hay ALTER TABLE posterior.

Ojo con
Los campos comunes son name, model, sku y description; los atributos propios de cada categoría viven en tablas aparte. No hay nada que garantice que el atributo de un producto pertenezca a la categoría de ese producto.

2.2.3. product_category_specification

Definida en 001_schema.sql:81.

Columna	Tipo	Nulabilidad	Default
id	SERIAL	NOT NULL (PK)	—
category_id	INTEGER	NOT NULL	—
name	VARCHAR(150)	NOT NULL	—
data_type	VARCHAR(50)	NOT NULL	—
unit	VARCHAR(50)	NULL	—
required	BOOLEAN	NOT NULL	FALSE
comparable	BOOLEAN	NOT NULL	TRUE

Columna	Tipo	Nulabilidad	Default
display_order	INTEGER	NOT NULL	0

Clave primaria
id

Claves foráneas
category_id → product_category.id, ON DELETE CASCADE. ON UPDATE no se indica (NO ACTION).

UNIQUE
No hay nada para category_id + name.

CHECK
Nada que limite los valores de data_type.

Índices
Solo la PK; no hay uno extra sobre category_id.

Triggers
Ninguno.

Quién la toca después
003_seed.sql:26; 005_demo_catalog.sql:39; 006_refrigeradores.sql:27,39; 007_Computacion.sql:46; 008_Telefonia.sql:28. No hay ALTER TABLE posterior.

Ojo con
data_type es texto libre sin CHECK, y el nombre de un atributo puede repetirse dentro de una misma categoría.

2.2.4. product_specification_value

Definida en 001_schema.sql:92.

Columna	Tipo	Nulabilidad	Default
id	SERIAL	NOT NULL (PK)	—
product_id	INTEGER	NOT NULL	—
specification_id	INTEGER	NOT NULL	—
value	VARCHAR(255)	NOT NULL	—

Clave primaria
id

Claves foráneas
product_id → product.id, ON DELETE CASCADE. specification_id → product_category_specificati
ON DELETE CASCADE. ON UPDATE no se indica en ninguna.

UNIQUE
(product_id, specification_id).

CHECK
No hay.

Índices
La PK y el índice del UNIQUE; nada más.

Triggers
Ninguno.

Quién la toca después
003_seed.sql:43; 005_demo_catalog.sql:133,221; 006_refrigeradores.sql:168,178,188,198,208,218; 007_Computacion.sql:226; 008_Telefonia.sql:147. No hay ALTER TABLE posterior.

Ojo con
value es VARCHAR(255) y nada en SQL revisa que el contenido calce con data_type. Tampoco hay una FK compuesta que impida asociar un producto con una especificación de otra categoría.

2.2.5. store

Definida en 001_schema.sql:104.

Columna	Tipo	Nulabilidad	Default
id	SERIAL	NOT NULL (PK)	—
name	VARCHAR(150)	NOT NULL	—
website_url	TEXT	NULL	—
logo_url	TEXT	NULL	—
rating	NUMERIC(3,2)	NULL	—
reputation	VARCHAR(100)	NULL	—
shipping_information	TEXT	NULL	—
general_conditions	TEXT	NULL	—
active	BOOLEAN	NOT NULL	TRUE
created_at	TIMESTAMP	NOT NULL	NOW()
updated_at	TIMESTAMP	NOT NULL	NOW()

Clave primaria
id

Claves foráneas
Ninguna.

UNIQUE / CHECK
No hay; en particular, name no es UNIQUE.

Índices
Solo la PK.

Triggers
Ninguno.

Quién la toca después
003_seed.sql:54; 005_demo_catalog.sql:72; 006_refrigeradores.sql:51; 007_Computacion.sql:68; 008_Telefonia.sql:47. No hay ALTER TABLE posterior.

Ojo con
El esquema deja repetir nombres de tienda. Hay que definir si la identidad de una tienda se basa en el nombre, el dominio web u otra cosa.

2.2.6. product_offer

Definida en 001_schema.sql:118.

Columna	Tipo	Nulabilidad	Default
id	SERIAL	NOT NULL (PK)	—

Columna	Tipo	Nulabilidad	Default
product_id	INTEGER	NOT NULL	—
store_id	INTEGER	NOT NULL	—
price	NUMERIC(12,2)	NOT NULL	—
list_price	NUMERIC(12,2)	NULL	—
currency	CHAR(3)	NOT NULL	'CLP'
shipping_cost	NUMERIC(12,2)	NULL	0
shipping_free	BOOLEAN	NOT NULL	FALSE
available	BOOLEAN	NOT NULL	TRUE
stock	INTEGER	NULL	—
condition	VARCHAR(20)	NOT NULL	'new'
product_url	TEXT	NULL	—
last_updated	TIMESTAMP	NOT NULL	NOW()

Clave primaria
id

Claves foráneas
product_id → product.id, ON DELETE CASCADE. store_id → store.id, ON DELETE CASCADE. ON UPDATE no se indica.

UNIQUE
(product_id, store_id).

CHECK
price >= 0; list_price >= 0; shipping_cost >= 0; stock >= 0; condition IN ('new','used','refurbis

Índices
PK; UNIQUE (product_id, store_id); idx_product_offer_product; idx_product_offer_store. Sobre product_url no hay ninguno.

Triggers
product_offer_currency_to_clp, BEFORE INSERT OR UPDATE, definido en 008_normalize_prices_to_c Llama a normalize_product_offer_currency_to_clp() (líneas 6-14), que le pone 'CLP' a NEW.currency. El mismo archivo actualiza las ofertas que ya existían (líneas 38-40). El comentario de las líneas 3-4 dice que se mantiene el monto numérico y solo se normaliza la etiqueta de moneda; no habla de convertir el importe.

Quién la toca después
003_seed.sql:59; 005_demo_catalog.sql:249; 006_refrigeradores.sql:300; 007_Computacion.sql:330; 008_Telefonia.sql:212; 008_normalize_prices_to_clp.sql:6-14,26-40. No hay ALTER TABLE posterior.

Ojo con
product_url admite NULL, no hay nada contra la cadena vacía y no tiene índice. La unicidad actual deja una sola oferta por producto y tienda, da igual la URL, la condición o la variante. La moneda queda forzada a CLP por el trigger. En esta tabla no hay columna de promoción ni de precio anterior.

2.2.7. product_price_history

Definida en 001_schema.sql:137.

Columna	Tipo	Nulabilidad	Default
id	SERIAL	NOT NULL (PK)	—
product_offer_id	INTEGER	NOT NULL	—
price	NUMERIC(12,2)	NOT NULL	—
is_promotional	BOOLEAN	NOT NULL	FALSE
recorded_at	TIMESTAMP	NOT NULL	NOW()

Clave primaria
id

Claves foráneas
product_offer_id → product_offer.id, ON DELETE CASCADE. Se relaciona con la oferta, no directo con producto o tienda.

UNIQUE
No hay.

CHECK
price >= 0.

Índices
PK y idx_product_price_history_offer. Por fecha o precio no hay ninguno.

Triggers
Ninguno.

Quién la toca después
003_seed.sql:68; 005_demo_catalog.sql:317; 006_refrigeradores.sql:336,350,364; 007_Computacion.s 008_Telefonia.sql:246. No hay ALTER TABLE, ni trigger de historial, ni protección contra UPDATE o DELETE.

Ojo con
Guarda precio, si era promocional y fecha; no guarda disponibilidad ni moneda. Y por el ON DELETE CASCADE, si se borra una oferta se va también todo su historial.

2.3. Resumen en una tabla

Tabla	PK	FKs	UNIQUE	Triggers
product_category	id	parent_category_id → product_category.id (SET NULL)	Ninguna	Ninguno
product	id	category_id → product_category.id (RESTRICT); brand_id → brand.id (SET NULL)	public_id, sku	Ninguno
product_category_specification	id	category_id → product_category.id (CASCADE)	Ninguna	Ninguno

Tabla	PK	FKs	UNIQUE	Triggers
product_specification_value	id	product_id → product.id (CASCADE); specification_id → product_category_specification.id (CASCADE)	product_id + specification_id	Ninguno
store	id	Ninguna	Ninguna (name no es único)	Ninguno
product_offer	id	product_id → product.id (CASCADE); store_id → store.id (CASCADE)	product_id + store_id	product_offer_currency_to_c
product_price_history	id	product_offer_id → product_offer.id (CASCADE)	Ninguna	Ninguno

2.4. Riesgos y dudas

Lo que está respaldado por evidencia:

- product_category.name no tiene UNIQUE (001_schema.sql:46-54) y store.name tampoco (001_schema.sql:104-116).
- product_offer tiene UNIQUE (product_id, store_id) (001_schema.sql:118-135).
- product_url admite NULL y no tiene CHECK contra cadena vacía (001_schema.sql:132).
- La moneda de product_offer se fuerza a 'CLP' con el trigger de 008_normalize_prices_to_clp.sql:6-1 y ese archivo también actualiza las ofertas existentes (líneas 38-40).
- El historial guarda precio, indicador promocional y fecha, y cuelga de la oferta con ON DELETE CASCADE (001_schema.sql:137-143).
- product_specification_value.value es VARCHAR(255) sin validación visible contra data_type, y no hay restricción compuesta que compare la categoría de la especificación con la del producto (001_schema.sql:81-98).
- La consulta de especificaciones trae nombre y valor uniendo por specification_id (products.sql:241-24 la del historial devuelve recorded_at, price e is_promotional (products.sql:252-261).
- El README habla de archivos 009 y 010 que no están en este checkout, así que no podemos confirmar si allí había protecciones adicionales.

Lo que falta decidir:

- *Identidad de la oferta:* ¿producto y tienda, URL u otra combinación?
- *URL:* ¿será obligatoria? ¿Cómo se normalizan parámetros de seguimiento y otros formatos?
- *Variantes y condiciones:* ¿condiciones distintas o publicaciones distintas de una misma tienda cuentan como ofertas separadas?
- *Identidad de la tienda:* ¿se evitan nombres duplicados? ¿Se identifica por nombre, dominio u otro campo?
- *Validación de atributos:* cómo se aplican tipo y obligatoriedad al crear o actualizar productos.

- *Retención del historial:* ¿debe sobrevivir si se borra la oferta, dado el CASCADE actual?
- *Cobertura del historial:* qué cambios deberían dejar registro y si todas las rutas de escritura lo cumplen.

2.5. Solo lectura y estado de Git

No se modificó, creó ni borró ningún archivo, no se ejecutó SQL ni migraciones, no se consultó ninguna base y no se levantó Docker. Al final, `git status -short` dio:

```
?? docs/database/clasificacion_sql_migraciones.txt
```

O sea que el árbol no estaba limpio: ese archivo figuraba como no rastreado. No lo tocamos. (Ojo: en los Pasos 3 y 4 ese archivo ya no aparece; queda como pregunta abierta.)

3. Paso 3: cómo se escriben productos y ofertas

3.1. Rutas HTTP

Los routers de productos y tiendas se registran en `backend/internal/http/router.go`; pricing e ingesta, en `backend/internal/features/router.go`. En todas estas rutas se ve CORS y ninguna autenticación; las de búsqueda son públicas en el router mostrado.

Método	Ruta	Handler	Dónde
GET	`/products`	`products.Handler.List`	`http/router.go:52`
GET	`/products/{publicID}`	`products.Handler.GetByPublicID`	`http/router.go:53`
POST	`/products`	`products.Handler.Create`	`http/router.go:54`
PUT	`/products/{publicID}`	`products.Handler.Update`	`http/router.go:55`
DELETE	`/products/{publicID}` (baja lógica)	`products.Handler.Deactivate`	`http/router.go:56`
GET	`/stores`	`stores.Handler.List`	`http/router.go:87`
GET	`/stores/{id}`	`stores.Handler.GetByID`	`http/router.go:88`
GET	`/offers/product/{productId}`	`features.Handler.OffersByProduct`	`features/router.go:18`
GET	`/offers/{id}/price-history`	`features.Handler.PriceHistory`	`features/router.go:19`
GET	`/offers/{id}`	`features.Handler.GetOffer`	`features/router.go:20`
POST	`/offers`	`features.Handler.CreateOffer`	`features/router.go:21`
PUT	`/offers/{id}`	`features.Handler.UpdateOffer`	`features/router.go:22`
POST	`/ingestion/products`	`features.Handler.IngestProducts`	`features/router.go:96`
POST	`/ingestion/offers`	`features.Handler.IngestOffers`	`features/router.go:97`
POST	`/ingestion/sync`	`features.Handler.SyncIngestion`	`features/router.go:98`
GET	`/search/products`	`features.Handler.SearchProducts`	`features/router.go:24`
GET	`/search/categories`	`features.Handler.SearchCategories`	`features/router.go:25`
GET	`/search/brands`	`features.Handler.SearchBrands`	`features/router.go:26`

3.2. Consultas en `backend/database/queries/products.sql`

En este archivo no hay consultas `:exec` ni `ON CONFLICT`.

Nombre sqlc	Tipo	Lee	Escribe	RET.	Líneas
`GetProductByID`	:one	product	—	No	1, 14-15
`GetProductByPublicID`	:one	product	—	No	18, 31-32
`GetProductDetailByPublicID`	:one	product, brand, product_category, product_rating_summary	—	No	36, 53-57
`ListProducts`	:many	product, brand, product_category, product_rating_summary	—	No	60, 77-82
`ListProductOffers`	:many	product_offer, store	—	No	85, 101-104
`CreateProduct`	:one	—	product	Sí	121-149
`UpdateProduct`	:one	product	product	Sí	152-174
`DeactivateProduct`	:one	product	product	Sí	177-194
`UpdateProductByPublicID`	:one	product	product	Sí	195-218
`DeactivateProductByPublicID`	:one	product	product	Sí	221-239

Nombre sqlc	Tipo	Lee	Escribe	RET.	Líneas
ListProductSpecifications	:many	product_specification_value, product_category_specification	—	No	241-249
ListProductPriceHistory	:many	product_price_history, product_offer	—	No	252-261

ListProductImages también está en products.sql (líneas 107-118), pero solo consulta product_image, que queda fuera de lo auditado.

3.3. Ficha de cada flujo de escritura

3.3.1. Ingesta de productos

Ruta: POST /ingestion/products → features.Handler.IngestProduct (features/router.go:96; features/ingestion.go:23).

Quién llama a quién.

- Si imageUrl viene con valor: IngestProduct → remoteJSON → POST PRODUCT_API_URL/products → products.Handler.Create → Repository.Create → sqlc CreateProduct y escritura de product_image (ingestion.go:29-45; products/handler.go:24-52; products/repository.go:24-69)
- Si imageUrl viene vacío: IngestProduct hace un INSERT INTO product directo, sin pasar por sqlc (ingestion.go:46-53).

1. **Qué recibe:** categoryId (int32), brandId opcional, name, model, sku, description, imageUrl y source (ingestion.go:12-21). El DTO del Product API no trae source (products/dto/product_reque
2. **Qué lee:** nada previo; no busca duplicados.
3. **Qué modifica y en qué orden:** por el Product API, product y luego product_image en una transacción (products/repository.go:25-31,41-63). Por la rama directa, solo product y sin transacción explícita (ingestion.go:47-50). La escritura directa de product_image en ingestion.go:55-56 no se puede alcanzar hoy.
4. **Validaciones en código:** exige categoryId > 0 y name no vacío tras TrimSpace (ingestion.go:25-26,4 El Product API rechaza JSON con campos desconocidos, recorta name e imageUrl y pide categoría positiva, nombre e imagen con URL HTTP(S) y host (products/handler.go:27-40).
5. **Restricciones de BD y errores:** product tiene category_id NOT NULL con FK, brand_id FK nullable, name NOT NULL y sku UNIQUE (001_schema.sql:56-69). Con INSERT directo, cualquier error da HTTP 400 genérico. Con el Product API, un error del repositorio da HTTP 500 genérico (products/handler.go:43-46), y la ingesta lo convierte en HTTP 502 (ingestion.go:40-42).
6. **Respuesta:** directo, HTTP 201 con productId, name y source (ingestion.go:61); remoto, HTTP 201 con el JSON del Product API (ingestion.go:44); errores, HTTP 400 (payload o INSERT directo) o 502.
7. **Transacción:** la rama directa no tiene; la del Product API sí (producto e imagen juntos).
8. **Historial:** no escribe product_price_history.
9. **Duplicados:** no mira SKU, nombre ni modelo antes de crear. Un SKU repetido termina en error genérico, no en actualización ni en ignorar.

10. **Ojo con:** hay dos formas de dar de alta un producto con distinto manejo de transacción y validación, el bloque de imagen directo parece código muerto y no hay reconciliación por SKU, nombre o modelo.

3.3.2. Ingesta de ofertas

Ruta: `POST /ingestion/offers` → `features.Handler.IngestOffer` (`features/router.go:97`; `features/ingestion.go:64`).

Quién llama a quién: `IngestOffer` → `GET STORE_API_URL/stores/{storeId}` → `POST PRICING_API_URL/of` → `features.Handler.CreateOffer` → `Service.ProductID` → `Repository.ResolveProductID` → INSERT en `product_offer` y `product_price_history` (`ingestion.go:64-81`; `catalog.go:71-102`; `service.go:18-22`; `repository.go:44-54`).

1. **Qué recibe:** `offerInput`: `productId` (string), `storeId` (int32), `price` (float64), `listPrice`, `currency`, `shippingCost`, `shippingFree`, `available`, `stock`, `condition` y `productUrl` (`catalog.go:14-2`
2. **Qué lee:** confirma la tienda preguntándole al Store API por ID; el Pricing API resuelve el producto activo por ID entero o por `public_id` UUID. No busca por SKU, nombre ni URL.
3. **Qué modifica:** el handler de ingesta no escribe directo; delega a `POST /offers`, que inserta `product_offer` y luego `product_price_history` en una transacción (`catalog.go:88-108`).
4. **Validaciones:** pide `productId` no vacío, `storeId > 0` y `price >= 0`, y consulta al Store API (`ingestion.go:66-78`). No se ve validación de URL, `currency`, trim, stock negativo ni `condition` en el handler de ingesta.
5. **Restricciones y errores:** FKs, CHECKs y UNIQUE de `product_offer` (`001_schema.sql:118-134`). Si falla la tienda o el Pricing API rechaza, HTTP 502 genérico (`ingestion.go:71-79`).
6. **Respuesta:** HTTP 201 con el JSON del Pricing API; HTTP 502 si falla la validación remota o el Pricing API.
7. **Transacción:** no hay una local que cubra las llamadas HTTP; el Pricing API sí transacciona oferta e historial.
8. **Historial:** al delegar en `POST /offers`, se inserta el precio inicial.
9. **Duplicados:** la identidad efectiva es `product_id + store_id` por el UNIQUE. No hay upsert ni búsqueda por URL; una oferta repetida rechaza el insert.
10. **Ojo con:** la tienda se resuelve por `storeId` y nunca se crea por nombre; verificar la tienda y escribir en pricing son dos llamadas separadas.

3.3.3. Crear oferta (`POST /offers`)

Ruta: `features.Handler.CreateOffer` (`features/router.go:21`; `catalog.go:71`). **Cadena:** `CreateOffer` → `Service.ProductID` → `Repository.ResolveProductID` → INSERT `product_offer` → INSERT `product_price_history`.

1. **Qué recibe:** los campos de `offerInput` (`catalog.go:14-26`).
2. **Qué lee:** `product`, para resolver el producto activo por `id` o `public_id`; no lee la oferta para deduplicar.
3. **Qué modifica:** `product_offer` y después `product_price_history`, en una transacción (`catalog.go:88-108`).
4. **Validaciones:** `storeId > 0` y `price >= 0`; resuelve `productId` a un producto activo; si `currency` viene vacía usa CLP y si `condition` viene vacía usa `new` (`catalog.go:73-87`). No

detecta que falte `price` en el JSON (un campo ausente se lee como cero) ni valida la URL ni una `currency` con valor.

5. **Restricciones y errores:** un error en el INSERT de oferta o historial da HTTP 400 "invalid store or offer data"; uno al iniciar o confirmar la transacción da HTTP 500 (`catalog.go:88-109`).
6. **Respuesta:** después del commit consulta la oferta con `h.one`: HTTP 200 si hay fila, 500 si falla la consulta, 404 si no hay fila (`catalog.go:112-117`; `handler.go:66-77`). Este POST no devuelve 201.
7. **Transacción:** sí, las dos escrituras, con rollback diferido.
8. **Historial:** siempre inserta el precio inicial, y `is_promotional` queda con el default FALSE (`catalog.go:101-102`).
9. **Duplicados:** no consulta ni actualiza una oferta existente; repetir `product_id + store_id` falla por UNIQUE. La URL no cuenta como identidad.
10. **Ojo con:** no se valida la tienda antes del INSERT (se confía en la FK). Y si falla la consulta de la respuesta después del commit, el cliente ve un error aunque las escrituras ya quedaron confirmadas.

3.3.4. Actualizar oferta (`PUT /offers/{id}`)

Ruta: `features.Handler.UpdateOffer` (`features/router.go:22`; `catalog.go:120`). **Cadena:** `UpdateOffer` → `SELECT price ... FOR UPDATE` → `UPDATE product_offer` → INSERT en `product_price_his` si cambió el precio (SQL inline en `catalog.go:120-165`).

1. **Qué recibe:** `offerInput`; se usa el id de la ruta y los campos de la oferta. `productId` y `storeId` del cuerpo no entran en el UPDATE.
2. **Qué lee:** `product_offer`, para bloquear la fila y sacar el precio anterior con FOR UPDATE (`catalog.go:139-144`).
3. **Qué modifica:** `product_offer` y, si cambió `price`, `product_price_history`, todo dentro de la transacción (`catalog.go:133-158`).
4. **Validaciones:** id válido, JSON decodificable y `price >= 0`; moneda vacía pasa a CLP y condición vacía a `new`. No valida `shippingCost`, `stock`, URL ni una `currency` con valor.
5. **Restricciones y errores:** los CHECK siguen aplicando. Error de lectura: HTTP 404; de UPDATE o del insert histórico: 400; de transacción: 500.
6. **Respuesta:** HTTP 200 con `{id, price, currency:"CLP", updated:true}` (`catalog.go:162-165`).
7. **Transacción:** sí; el SELECT FOR UPDATE, el UPDATE y el posible historial se confirman juntos.
8. **Historial:** solo si `oldPrice != currentPrice`. Cambiar otros campos por sí solos no deja registro (`catalog.go:151-152`).
9. **Duplicados:** no cambia producto ni tienda y no busca duplicados.
10. **Ojo con:** la respuesta fija la moneda en CLP; el trigger de la BD también la fuerza.

3.3.5. Crear producto (`POST /products`)

Cadena: `products.Handler.Create` → `Repository.Create` → sqlc `CreateProduct` → INSERT `product_image` (`http/router.go:54`; `products/handler.go:24-52`; `products/repository.go:41-69`).

1. **Qué recibe:** `categoryId`, `brandId`, `name`, `model`, `sku`, `description` e `imageUrl` (`product_request.go:8-`

2. **Qué lee:** nada previo; no busca duplicados.
3. **Qué modifica:** `product` y después `product_image`, en una transacción (`repository.go:25-63`).
4. **Validaciones:** rechaza campos desconocidos, recorta `name` e `imageUrl`, y pide `categoryId > 0`, nombre y URL HTTP(S) con host (`handler.go:27-40`).
5. **Restricciones y errores:** FK, NOT NULL y SKU UNIQUE. Cualquier error del repositorio, incluso uno de integridad, responde HTTP 500 genérico (`handler.go:43-46`).
6. **Respuesta:** 201 con el producto; 400 si el payload es inválido; 500 si falla el repositorio.
7. **Transacción:** sí; si falla `product_image`, no se confirma nada.
8. **Historial:** no genera.
9. **Duplicados:** no deduplica; un SKU repetido lo rechaza el UNIQUE.
10. **Ojo con:** una colisión de SKU sale como 500 en vez de un conflicto de datos.

3.3.6. Actualizar producto (`PUT /products/{publicID}`)

Cadena: `products.Handler.Update` → `Repository.UpdateByPublicID` → sqlc `UpdateProductByPublicID` (`http/router.go:55`; `handler.go:208-239`; `products.sql:195-218`).

1. **Qué recibe:** el UUID público en la ruta y, en el cuerpo, categoría, marca, nombre, modelo, SKU y descripción (`product_request.go:18-25`).
2. **Qué lee:** `product`, dentro del UPDATE filtrado por `public_id` y `active=true`.
3. **Qué modifica:** `product`; UPDATE con `updated_at=now()` y RETURNING.
4. **Validaciones:** valida UUID y JSON y rechaza campos desconocidos. No hay chequeo previo de nombre vacío ni de `categoryId > 0`. Modelo, SKU y descripción vacíos se guardan como NULL.
5. **Restricciones y errores:** aplican FK, NOT NULL y UNIQUE. Cualquier error del repositorio se responde HTTP 404 "product not found", incluso un SKU duplicado (`handler.go:228-234`).
6. **Respuesta:** el JSON del producto con HTTP 200 implícito; UUID inválido da 400; error del repositorio, 404.
7. **Transacción:** no es explícita; es una sola sentencia.
8. **Historial:** no toca ofertas ni historial.
9. **Duplicados:** no deduplica; un SKU repetido puede fallar por UNIQUE y responder 404.
10. **Ojo con:** una colisión de SKU se puede confundir con un producto que no existe.

3.3.7. Baja de producto (`DELETE /products/{publicID}`)

Cadena: `products.Handler.Deactivate` → `Repository.DeactivateByPublicID` → sqlc `DeactivateProductB` (`http/router.go:56`; `handler.go:242`; `products.sql:221-239`).

- **Qué recibe:** el UUID público en la ruta. **Qué lee:** `product`, dentro del UPDATE filtrado por `public_id` y `active=true`.
- **Qué modifica:** pone `product.active=false` y `updated_at=now()`; no hace DELETE.
- **Validaciones:** un UUID inválido da 400; en la ruta no se ve autenticación. Los errores del repositorio se responden como 404.
- **Respuesta:** devuelve el producto desactivado. **Transacción:** una sola sentencia.

- **Historial:** no crea fila; como es baja lógica, no dispara el ON DELETE CASCADE.
- **Ojo con:** ofertas e historial siguen asociados mientras no haya borrado físico.

3.3.8. Otros que escriben: tiendas y especificaciones

No encontré: ningún handler ni repository que inserte o actualice `store`, `product_specification_value` o `product_category_specification`. Las rutas de tiendas son solo GET (`http/router.go:87-88`) y el repository de tiendas tiene solo métodos de lectura (`stores/repository.go:18` en adelante).

3.4. Respuestas puntuales

Ingesta de productos
Confirmado: con `imageUrl` vacío inserta directo en `product`; si no, delega al Product API (`ingestion.go:29-48`). **Confirmado:** no busca por SKU, nombre ni modelo antes de crear; SKU es UNIQUE, nombre y modelo no. **Confirmado:** categoría y marca llegan como IDs y no se crean; los DTOs no traen especificaciones.

Ingesta de ofertas
Confirmado: consulta al Store API por `storeId` y después delega al Pricing API. **Confirmado:** la identidad efectiva es `product_id + store_id`; no hay upsert ni identificación por URL. **Confirmado:** la tienda se busca por ID, no por nombre, y no se crea. **Confirmado:** `store.name` no es UNIQUE. **No encontré:** búsqueda de tienda por nombre.

Normalización de `product_url`
Confirmado: el código pasa `product_url` al INSERT o UPDATE tal cual, sin trim, sin minúsculas, sin quitar parámetros de seguimiento ni la barra final (`catalog.go:96-100,146-150`). **Confirmado:** no se valida la URL, y la cadena vacía no se convierte en NULL.

Moneda
Confirmado: el código pone CLP solo si `currency` viene vacía (`catalog.go:82-87,127-132`). **Confirmado:** el trigger fuerza `NEW.currency='CLP'` en INSERT y UPDATE (`008_normalize_prices_to_clp.s`

`list_price`
Confirmado: se recibe, se inserta o actualiza y se devuelve. **No encontré:** algún cálculo de descuento o precio anterior basado en `list_price`.

Timestamps
Confirmado: `product_offer.last_updated` tiene default NOW() y `UpdateOffer` lo fija con NOW(). **Confirmado:** las consultas de actualización y baja de producto ponen `updated_at=now()`. **No encontré:** triggers de timestamps.

Historial al crear
Confirmado: `CreateOffer` inserta una fila histórica con el primer precio, en la misma transacción.

Borrado
Confirmado: no hay `DELETE /offers/{id}` en el router de pricing. **Confirmado:** `DELETE /products/{publicI` es una baja lógica. **Confirmado:** si se borrara una oferta físicamente, el CASCADE se llevaría su historial. **No encontré:** un flujo HTTP de borrado físico de ofertas.

`ingestion_run`
Confirmado: solo `POST /ingestion/sync` inserta en `ingestion_run`, con estado `accepted` (`ingestion.go:89-` `004_platform_apis.sql:57`). La ingesta de productos y ofertas no la usa.

Especificaciones
Confirmado: no hay flujos que escriban valores ni validen `data_type` o `required`; la única consulta encontrada solo lee `product_specification_value` (`products.sql:241-249`).

`product_url`
Confirmado: se lee y se devuelve en listados y detalle de ofertas. **No encontré:** una consulta que busque ofertas por `product_url`.

3.5. sqlc y pruebas

sqlc. Se configura en `backend/sqlc.yaml` (líneas 1-12): engine `postgresql`, consultas en `database/queries`, esquema en `database/migrations` (la carpeta entera), salida Go en `internal/database/generated`, paquete `generated`, driver `pgx/v5` y `emit_json_tags: true`. Los archivos generados están versionados (`db.go`, `models.go`, `products.sql.go`, `services.sql.go`, `stores.sql.go`).

Migraciones. Confirmado: siguen los prefijos repetidos 006, 007 y 008 (ver Paso 1). **Riesgo:** el prefijo por sí solo no fija un orden claro entre archivos con el mismo número, si el orden entre ellos importa. No corrimos migraciones ni sqlc.

Pruebas. Buscando `backend/**/*_test.go` dieron 0 archivos. No hay pruebas de duplicados, historial, validaciones ni de integración con la BD.

3.6. Cuándo se genera historial

Evento	¿Deja fila en el historial?	Dónde
Alta de oferta	Sí, con el precio inicial	`catalog.go:96-108`
Cambio de `price`	Sí, solo si el precio nuevo es distinto	`catalog.go:139-152`
Cambio de `is_promotional`	No hay campo ni actualización en `offerInput`; las filas nuevas usan FALSE	`catalog.go:14-26,101-102`
Cambio de `list_price`	No por sí solo	`catalog.go:146-152`
Cambio de `available`	No por sí solo	`catalog.go:147-152`
Cambio de `stock`	No por sí solo	`catalog.go:147-152`
Cambio de `product_url`	No por sí solo	`catalog.go:148-152`
Cambio de `condition`	No por sí solo	`catalog.go:147-152`
Cambio de `shipping_cost`	No por sí solo	`catalog.go:147-152`
Ingesta de una oferta que ya existe, sin cambios	No: sin upsert, el duplicado falla por UNIQUE	`catalog.go:96-100`
Borrado de oferta	No deja fila; un borrado físico elimina el historial por CASCADE, y no hay ruta HTTP de borrado	`features/router.go:17-22`

3.7. Riesgos y dudas

Lo confirmado:

- Los errores de integridad de `POST /offers` salen como 400, pero la ingesta que delega al Pricing API los convierte en 502.
- Los errores de escritura de `POST /products` salen como 500 genérico, y en el `PUT` de producto cualquier error del repositorio sale como 404.
- El alta de oferta genera historial inicial; las actualizaciones solo cuando cambia el precio.

- No hay deduplicación por URL ni flujo que escriba especificaciones o tiendas.
- Al final de este paso, Git estaba limpio y el archivo no rastreado del Paso 2 no aparecía.

Suposiciones y preguntas:

- *Suposición:* si se reintenta una ingesta de oferta ya insertada, lo más probable es que falle por UNIQUE en vez de actualizar (no hay upsert visible).
- ¿Una colisión de SKU debería responder 409 en vez de 500 o 404?
- ¿La URL vacía debe guardarse como NULL o como cadena vacía?
- ¿El historial debe registrar solo cambios de precio o también disponibilidad, promoción y otros datos?

3.8. Solo lectura y estado de Git

No modificamos, creamos ni borramos archivos, no ejecutamos SQL ni migraciones, no levantamos servidor ni Docker, no consultamos bases y no corrimos `go test` ni `go build`. `git status -short`: sin salida (árbol limpio).

4. Paso 4: línea base de pruebas y compilación

4.1. Cerrando el Paso 3

4.1.1. Estado real de Git

```
> git status --short
(sin salida)
> git branch --show-current
feature/services-cart-and-comparison-new
> Test-Path .\docs\database\clasificacion_sql_migraciones.txt
False
```

Confirmado: el archivo no rastreado que se mencionó en el contexto no existe en esa ruta y el árbol estaba limpio.

4.1.2. Archivos de prueba

`Get-ChildItem .\backend -Recurse -File -Filter *_test.go` devolvió 0 archivos. No hay pruebas que revisar ni build tags de integración.

4.1.3. Código que parece inalcanzable en `ingestion.go`

Archivo `backend\internal \features \ingestion .go`:

```
 23: func (h *Handler) IngestProduct(w http.ResponseWriter, r *http.Request) {
 24:     var input ingestionProductInput
 25:     if !decode(r, &input) || input.CategoryID <= 0 || strings.TrimSpace(input.Name) == "" {
 26:         badRequest(w, "categoryId and name are required")
 27:         return
 28:     }
 29:     if input.ImageURL != "" {
 30:         productRequest := map[string]any{
 31:             "categoryId":  input.CategoryID,
 32:             "brandId":     input.BrandID,
 33:             "name":        input.Name,
 34:             "model":       input.Model,
 35:             "sku":         input.SKU,
 36:             "description": input.Description,
 37:             "imageUrl":    input.ImageURL,
 38:         }
 39:         var product json.RawMessage
 40:         if err := remoteJSON(r.Context(), "PRODUCT_API_URL", "http://localhost:8081", http.
    MethodPost, "/products", productRequest, &product); err != nil {
 41:             apiutil.WriteError(w, http.StatusBadGateway, "product API rejected the ingested
    product")
 42:             return
 43:         }
 44:         apiutil.WriteJSON(w, http.StatusCreated, product)
 45:         return
 46:     }
 47:     var publicID string
 48:     err := h.db.QueryRow(r.Context(), `INSERT INTO product (category_id,brand_id,name,model,
    sku,description)
 49:         VALUES (1,2,3,NULLIF(4,''),NULLIF(5,''),NULLIF(6,'')) RETURNING public_id::text`,
 50:         input.CategoryID, input.BrandID, strings.TrimSpace(input.Name), input.Model, input.
    SKU, input.Description).Scan(&publicID)
 51:     if err != nil {
 52:         apiutil.WriteError(w, http.StatusBadRequest, "could not ingest product; check
    category, brand, and SKU")
```

```
 53:         return
 54:     }
 55:     if input.ImageURL != "" {
 56:         if _, err := h.db.Exec(r.Context(), `INSERT INTO product_image(product_id,image_url,
    alt_text) SELECT id,2,3 FROM product WHERE public_id=$1::uuid`, publicID, input.ImageURL,
    input.Name); err != nil {
 57:             apiutil.WriteError(w, http.StatusInternalServerError, "product was saved but
    image could not be saved")
 58:             return
 59:         }
 60:     }
 61:     apiutil.WriteJSON(w, http.StatusCreated, map[string]any{"productId": publicID, "name":
    input.Name, "source": defaultSource(input.Source)})
 62: }
```

Confirmado: si `imageUrl` trae algo, se usa el Product API y la función retorna en la línea 45. Si viene vacío, la condición de la línea 55 nunca se cumple. O sea, el bloque de las líneas 55-59 no se puede alcanzar con el flujo actual.

Ojo: en los fragmentos de código de este paso, los marcadores de parámetros SQL (`$1`, `$2`, ...) aparecen sin el signo `$` y en algunas sentencias faltan espacios. Los dejamos tal cual venían en el informe original; probablemente se perdió algo al copiar.

4.1.4. Chequeo por muestreo de líneas citadas en el Paso 3

Todas las citas que cotejamos coinciden con el contenido y la numeración actuales: `ingestion.go:29-53`, `catalog.go:88-108`, `catalog.go:139-152`, `features/router.go:17-26`, `features/router.go:96-98`, `internal/http/router.go:52-56` e `internal/http/router.go:87-88`.

```
catalog.go:88-108
 88:     tx, err := h.db.Begin(r.Context())
 89:     if err != nil {
 90:         apiutil.WriteError(w, http.StatusInternalServerError, "could not start offer
    transaction")
 91:         return
 92:     }
 93:     defer tx.Rollback(r.Context())
 94:     var offerID int32
 95:     var savedPrice float64
 96:     err = tx.QueryRow(r.Context(), `INSERT INTO product_offer
 97:         (product_id, store_id, price, list_price, currency, shipping_cost, shipping_free,
    available, stock, condition, product_url)
 98:         VALUES (1,2,3,4,5,6,7,8,9,10,$11) RETURNING id, price`, productID, input.StoreID,
 99:         input.Price, input.ListPrice, input.Currency, input.ShippingCost, input.ShippingFree
    ,
100:         input.Available, input.Stock, input.Condition, input.ProductURL).Scan(&offerID, &
    savedPrice)
101:     if err == nil {
102:         _, err = tx.Exec(r.Context(), `INSERT INTO product_price_history (product_offer_id,
    price) VALUES (1,2)`, offerID, savedPrice)
103:     }
104:     if err != nil {
105:         apiutil.WriteError(w, http.StatusBadRequest, "invalid store or offer data")
106:         return
107:     }
108:     if err = tx.Commit(r.Context()); err != nil {

catalog.go:139-152
139:     var oldPrice float64
140:     err = tx.QueryRow(r.Context(), `SELECT price FROM product_offer WHERE id = $1 FOR UPDATE
    `, offerID).Scan(&oldPrice)
141:     if err != nil {
```

```
 142:         apiutil.WriteError(w, http.StatusNotFound, "offer not found")
 143:         return
 144:     }
 145:     var currentPrice float64
 146:     err = tx.QueryRow(r.Context(), `UPDATE product_offer SET price=2,listprice=3, currency=
    $4,
 147:         shipping_cost=5,shippingfree=6, available=7,stock=8, condition=$9,
 148:         product_url=10,lastupdated=NOW()WHEREid=1 RETURNING price`, offerID, input.Price,
    input.ListPrice,
 149:         input.Currency, input.ShippingCost, input.ShippingFree, input.Available, input.Stock
    ,
 150:         input.Condition, input.ProductURL).Scan(&currentPrice)
 151:     if err == nil && oldPrice != currentPrice {
 152:         _, err = tx.Exec(r.Context(), `INSERT INTO product_price_history (product_offer_id,
    price) VALUES (1,2)`, offerID, currentPrice)

features/router.go:17-26
  17:     case "pricing-api":
  18:         r.Get("/offers/product/{productId}", h.OffersByProduct)
  19:         r.Get("/offers/{id}/price-history", h.PriceHistory)
  20:         r.Get("/offers/{id}", h.GetOffer)
  21:         r.Post("/offers", h.CreateOffer)
  22:         r.Put("/offers/{id}", h.UpdateOffer)
  23:     case "search-api":
  24:         r.Get("/search/products", h.SearchProducts)
  25:         r.Get("/search/categories", h.SearchCategories)
  26:         r.Get("/search/brands", h.SearchBrands)

features/router.go:96-98
  96:         r.Post("/ingestion/products", h.IngestProduct)
  97:         r.Post("/ingestion/offers", h.IngestOffer)
  98:         r.Post("/ingestion/sync", h.SyncIngestion)

internal/http/router.go:52-56
  52:     r.Get("/products", productHandler.List)
  53:     r.Get("/products/{publicID}", productHandler.GetByPublicID)
  54:     r.Post("/products", productHandler.Create)
  55:     r.Put("/products/{publicID}", productHandler.Update)
  56:     r.Delete("/products/{publicID}", productHandler.Deactivate)

internal/http/router.go:87-88
  87:     r.Get("/stores", storeHandler.List)
  88:     r.Get("/stores/{id}", storeHandler.GetByID)
```

4.1.5. Autenticación

Confirmado: sí hay referencias de autenticación y autorización en el backend: `features/handler.go:23` (middleware `requireSession`), `features/auth.go:138` (extracción del token Bearer) e `internal/http/router.` (chequeo de token en una ruta de usuarios).

Confirmado: `requireSession` se aplica a los grupos de historial y búsquedas guardadas, favoritos y carrito (`features/router.go:27-34,55-74`). En las rutas de productos, ofertas e ingesta que revisamos no aparece: `NewProductRouter` registra las de producto directo (`http/router.go:42-56`) y las de pricing e ingesta también (`features/router.go:10-22,95-98`).

4.2. Chequeos antes de correr

- Go: `go version go1.27.1 windows/amd64`; `backend\go .mod` dice `go 1.27.1` y no tiene directiva `toolchain`. Coinciden.

- GOFLAGS: vacío; GOMODCACHE: C:\Users \cc 077\go \pkg \mod ; GOPROXY: https://proxy.golang.org,
- Variables de entorno (sin mostrar valores): DATABASE_URL=False, POSTGRES_URL=False, PGHOST=False.
- No hay archivos *_test.go ni pruebas que lean variables con os.Getenv, así que no se ve dependencia de BD, red o Docker.
- Decisión: era seguro correr los comandos pedidos.

4.3. Línea base

Comando	Exit	Resultado	OK	FAIL	Sin pruebas	Errores
go test -mod=readonly ./...	0	Correcto (˜12,1 s)	0 con tests	0	38	Ninguno
go build -mod=readonly ./...	0	Correcto (˜8,8 s)	38 compilados	0	n/a	Ninguno

Los 38 paquetes dijeron [no test files] (todos bajo github.com/DoSafills/SOLOServis/backend/):

```
cmd/auth-api            cmd/cart-api            cmd/comparison-api
cmd/favorites-api       cmd/ingestion-api       cmd/notification-api
cmd/pricing-api         cmd/product-api         cmd/review-api
cmd/search-api          cmd/server              cmd/service-api
cmd/store-api           cmd/user-api            cmd/watchlist-api
internal/apiutil        internal/auth           internal/cart
internal/comparisons    internal/config         internal/database
internal/database/generated                     internal/favorites
internal/features       internal/http           internal/ingestion
internal/notifications  internal/pricing        internal/products
internal/products/dto   internal/reviews        internal/runner
internal/search         internal/services       internal/services/dto
internal/stores         internal/stores/dto     internal/watchlist
```

4.4. Fallos que ya existían

No hubo fallos ni en go test ni en go build, así que no hay nada que clasificar como previo.

4.5. Ojo con el alcance

Que go test pase no prueba nada funcional: no hay archivos de prueba y todos los paquetes dijeron [no test files]. Y que go build compile no demuestra:

- que las migraciones se puedan aplicar o sean compatibles con una base existente;
- que el esquema que interpreta sqlc coincida con el real;
- que los flujos de ofertas e historial funcionen contra una base de datos;
- que estén cubiertos los duplicados, las validaciones o los errores de integridad.

4.6. Dónde quedaron los logs

Fuera del repositorio:

- C:\Users \cc 077\AppData \Local \Temp \dia 1_paso4_20261017_003813\go _test.txt
- C:\Users \cc 077\AppData \Local \Temp \dia 1_paso4_20261017_003813\go _build.txt

4.7. Preguntas nuevas

- El archivo `docs/database/clasificacion_sql_migraciones.txt` que se nombró en el contexto no existe en la ruta revisada y el árbol está limpio. ¿El contexto era de otro worktree o de otro momento?
- No hay pruebas. ¿La línea base esperada del Paso 4 es solo comprobar que compila, hasta que se agreguen tests?

4.8. Cierre

No se modificó ningún archivo del repositorio, no se ejecutó SQL ni migraciones, no se levantaron servidor ni Docker y no se consultó ninguna base. Verificación final: `git status -short`, `git diff -stat` y `git diff -name-only - backend/go.mod backend/go.sum` sin salida.

5. Paso 5: diagnóstico en bases desechables

5.1. Antes de empezar

- **Confirmado:** `docker -version`: `Docker version 29.7.2, build a7dcaa6`.
- **Confirmado:** `docker ps -a` mostró otros contenedores que ya existían; no los tocamos.
- **Confirmado:** los nombres `soloservis_audit_tmp` y `soloservis_audit_tmp2` estaban libres y los puertos 55432 y 55433 no tenían conexiones (`Get-NetTCPConnection`).
- **Confirmado:** rama actual `feature/services-cart-and-comparison-new` y Git sin cambios.

Antes de correr `docker run` pedimos confirmación y llegó: "Confirmo: ejecutar los dos comandos". Armamos las bases desechables A y B con tmpfs y puertos publicados solo en 127.0.0.1. `pg_isready` respondió bien en el segundo intento en cada contenedor.

5.2. Aplicando las migraciones

Aplicamos las 11 migraciones una por una en ambas bases. Cada `psql` con `ON_ERROR_STOP=1` terminó con código 0 y sin errores.

Archivo	Orden A	Orden B	Error
`001_schema.sql`	OK (0)	OK (0)	Ninguno
`002_views.sql`	OK (0)	OK (0)	Ninguno
`003_seed.sql`	OK (0)	OK (0)	Ninguno
`004_platform_apis.sql`	OK (0)	OK (0)	Ninguno
`005_demo_catalog.sql`	OK (0)	OK (0)	Ninguno
`006_demo_catalog_services.sql`	OK (0)	OK (0)	Ninguno
`006_refrigeradores.sql`	OK (0)	OK (0)	Ninguno
`007_Computacion.sql`	OK (0)	OK (0)	Ninguno
`007_service_cart.sql`	OK (0)	OK (0)	Ninguno
`008_Telefonia.sql`	OK (0)	OK (0)	Ninguno
`008_normalize_prices_to_clp.sql`	OK (0)	OK (0)	Ninguno

En B dimos vuelta el orden dentro de cada pareja: `006_refrigeradores` antes de `006_demo_catalog_services`; `007_service_cart` antes de `007_Computacion`; `008_normalize_prices_to_clp` antes de `008_Telefonia`.

5.3. Qué pasa con los prefijos repetidos

Confirmado: en los dos órdenes todo se aplicó sin errores, y los conteos de filas, monedas y condiciones salieron iguales en A y B.

Hasta dónde llega esta conclusión: en esta prueba no vimos diferencias para los pares 006, 007 y 008. Eso vale para estas migraciones, estas métricas y estas bases demo; no prueba que sean equivalentes en cualquier estado o consulta.

5.4. Consultas de diagnóstico

Ojo: todo esto sale de datos demo en bases desechables con tmpfs. No dice nada de una base real o compartida. Las 20 consultas corrieron en A, sin errores, cada una dentro de `BEGIN READ ONLY` y `ROLLBACK`.

N.º	Pregunta	Resultado	Estado
1	URLs repetidas por tienda	Ninguna	OK, sin filas
2	URLs nulas o vacías, total y por tienda	Total 0; sin filas por tienda	OK
3	URLs con espacios al inicio o final	Ninguna	OK, sin filas
4	URLs con parámetros de seguimiento o barra final	3 con parámetros; 0 con barra final; 0 con ambos	OK
5	Misma URL en distintas tiendas	Ninguna	OK, sin filas
6	Tiendas con nombre repetido (ignorando mayúsculas y espacios)	Ninguna	OK, sin filas
7	Tiendas con el mismo dominio de website URL	Ninguna	OK, sin filas
8	Categorías con nombre repetido bajo el mismo padre	Ninguna	OK, sin filas
9	Especificaciones repetidas por categoría y nombre	Ninguna	OK, sin filas
10	Productos con SKU nulo y con nombre+modelo repetidos	SKU nulo: 0; pares repetidos: ninguno	OK
11	Valores de especificación de otra categoría que la del producto	Ninguno	OK, sin filas
12	Valores incompatibles con `number` o `boolean` según patrones	Ninguno	OK, sin filas
13	Ofertas sin fila de historial	4: IDs 2, 4, 5 y 6	OK
14	Ofertas con varios historiales y precios consecutivos iguales	48 con varias filas; 0 ofertas y 0 pares con precios consecutivos iguales	OK
15	Último precio histórico distinto del precio actual	Ninguna	OK, sin filas
16	Conteo por moneda y condición	CLP: 71; new: 69; refurbished: 1; used: 1	OK
17	`list_price` menor que `price`	Ninguna	OK, sin filas
18	No disponibles o con stock cero	2 con stock cero (IDs 47 y 51); ninguna con `available=FALSE`	OK
19	Conteo por tabla	product: 35; product_offer: 71; store: 7; product_price_history: 115; product_specification_value: 98	OK
20	Ofertas que romperían UNIQUE `(store_id, product_url)`	0 grupos duplicados; 0 filas de más	OK

5.5. Revisando el esquema en A

Confirmado: al inspeccionar las tablas y consultar `information_schema` y `pg_indexes` vimos que:

- product_offer tiene índices para su PK, product_id y store_id, más el UNIQUE (product_id, store_id). Para product_url no hay.
- product_offer_currency_to_clp está instalado para INSERT y UPDATE.
- product_price_history tiene PK, índice por product_offer_id, CHECK de precio no negativo y FK a product_offer con ON DELETE CASCADE. No hay trigger de historial.
- cart_item incluye product_id, offer_id y service_offer_id, con restricciones para elegir entre producto o servicio y sus FKs.
- cart_item tiene PK, UNIQUE (user_id, offer_id), un índice por usuario y fecha, y un índice UNIQUE parcial por (user_id, service_offer_id) cuando service_offer_id IS NOT NULL.

Comparado con el Paso 2, calza: la unicidad producto-tienda, la falta de índice para la URL, el trigger CLP y el CASCADE del historial. Además confirmamos el índice del historial y la estructura de cart_item después de 007_service_cart.

5.6. Archivos de trabajo

Quedaron fuera del repositorio, en C:\Users \cc 077\AppData \Local \Temp \dia 1_paso5_20261017_0115 consultas.sql, resultados.txt, verificacion_esquema.sql, verificacion_esquema.txt, comparacion_AB comparacion_A.txt, comparacion_B.txt, migraciones_A.csv, migraciones_B.csv, A_*.sql.log y B_*.sql.log (los logs de cada migración están en la misma carpeta).

5.7. Limpieza

Confirmado: docker rm -f -v soloservis_audit_tmp y docker rm -f -v soloservis_audit_tmp2 terminaron con código 0. Verificación: AUDIT_CONTAINERS_REMAINING=none.

Los contenedores que ya existían siguen intactos (no usamos docker compose):

```
soloservis_cart_api          soloservis_pricing_api       soloservis_service_api
soloservis_product_api       soloservis_notification_api  soloservis_watchlist_api
soloservis_ingestion_api     soloservis_comparison_api    soloservis_review_api
soloservis_search_api        soloservis_favorites_api     soloservis_store_api
soloservis_auth_api          soloservis_user_api          soloservis_postgres
```

5.8. Hallazgos nuevos y preguntas

Lo confirmado:

- En los datos demo hay 4 ofertas sin historial, aunque el flujo de alta intenta crear el historial inicial.
- 48 ofertas tienen más de una fila de historial.
- No hay precios consecutivos iguales en el historial.
- El último precio histórico coincide siempre con el precio actual.
- Las 71 ofertas están en CLP en ambas bases.
- No hay duplicados en (store_id, product_url) en los datos demo.

Pregunta abierta: ¿el historial debe registrar solo cambios de precio o también disponibilidad, promoción y otros datos?

Pendiente: nada de esto reemplaza una revisión sobre una base real; eso sigue esperando autorización del equipo.

5.9. Cierre

No consultamos ninguna base real ni compartida, no usamos `docker compose` ni el volumen o la base del proyecto, las únicas bases consultadas fueron los dos contenedores desechables (ya borrados) y no se tocó ningún archivo del repositorio. `git status -short` y `git diff -stat`: sin salida.

6. Paso 6: Kubernetes y despliegue (revisión estática)

Alcance: lectura de archivos y consultas Git. No se consultó ningún cluster. El informe original no trae fecha; solo queda indicada en el nombre de la carpeta temporal donde se guardó.

Criterio. Confirmado: es lo que se ve en un archivo, una rama o la salida de una lectura de Git. *Suposición* es una inferencia explícita que no se validó contra un cluster. No copiamos valores de Secrets en este informe.

6.1. Preparación

- Rama actual: `feature/services-cart-and-comparison-new`. Commits observados: HEAD `2738771` y `origin/feature/k8s-mvp-deploy` en `9f865e9`.
- Git informó que `origin/feature/k8s-mvp-deploy` es ancestro de la rama actual. `git status -short` y `git diff -stat` no dieron salida.
- Las rutas `k8s/`, `scripts/k8s/deploy.ps1` y `docker/` no tienen diferencias entre las dos ramas: los manifests y el script son idénticos, y la rama actual es descendiente de la k8s. El script no fija una rama de despliegue; si se ejecutara, tomaría los archivos del checkout en el que esté.
- La diferencia importante está en `backend/database/migrations`: la rama k8s tiene 8 archivos y la actual tiene 11. Los manifests son los mismos, pero el conjunto de SQL que copia el script cambia según el checkout.
- El overlay `k8s/overlays/local` no existe en el checkout. El `.gitignore` lo excluye, junto con los kubeconfigs (`.gitignore:49-51`).
- No se cambió de rama. No se ejecutó `kubectl`, Docker, SQL ni `deploy.ps1`, y no se conectó ningún cluster ni base.

6.2. Inventario

6.2.1. Archivos de Kubernetes y script

Presentes en las dos ramas comparadas.

Ruta	Bytes
k8s/README.md	2702
k8s/base/ingress.yaml	975
k8s/base/kustomization.yaml	1466
k8s/base/frontend/deployment.yaml	2219
k8s/base/frontend/service.yaml	318
k8s/base/migrations/job.yaml	2584
k8s/base/migrations/migrate.sh	2160
k8s/base/postgres/secret.yaml	413
k8s/base/postgres/service.yaml	794
k8s/base/postgres/statefulset.yaml	3207
k8s/base/product-api/deployment.yaml	3003
k8s/base/product-api/service.yaml	332
k8s/base/store-api/deployment.yaml	2987
k8s/base/store-api/service.yaml	324
scripts/k8s/deploy.ps1	39756

6.2.2. Carpeta `docker/`

También presente en ambas ramas.

Ruta	Bytes
`docker/compose.yml`	10495 (solo metadatos; no se leyó)
`docker/api/Dockerfile`	383
`docker/auth-api/Dockerfile`	364
`docker/cart-api/Dockerfile`	364
`docker/comparison-api/Dockerfile`	394
`docker/favorites-api/Dockerfile`	389
`docker/frontend/Dockerfile`	1527
`docker/frontend/Dockerfile.dockerignore`	230
`docker/frontend/nginx.conf`	1921
`docker/ingestion-api/Dockerfile`	389
`docker/notification-api/Dockerfile`	404
`docker/pricing-api/Dockerfile`	379
`docker/product-api/Dockerfile`	715
`docker/review-api/Dockerfile`	374
`docker/search-api/Dockerfile`	374
`docker/service-api/Dockerfile`	405
`docker/store-api/Dockerfile`	703
`docker/user-api/Dockerfile`	390
`docker/watchlist-api/Dockerfile`	389

El script de despliegue solo construye `product-api`, `store-api` y `frontend` (`deploy.ps1:555-571`). No inspeccionamos el contenido de `docker/compose.yml`.

6.2.3. Recursos

En la base, el namespace declarado es `student-rrios` (`k8s/base/kustomization.yaml:9`); el overlay generado lo reemplaza según el kubeconfig (`deploy.ps1:210-215`). La etiqueta `app.kubernetes.io/part` la agrega la Kustomization (`kustomization.yaml:11-13`). Los ConfigMaps generados pueden llevar un sufijo hash de Kustomize, y sus referencias se reescriben al renderizar.

Tipo	Nombre	Namespace base	Archivo:líneas
Ingress	`soloservis`	`student-rrios`	`k8s/base/ingress.yaml:8-25`
StatefulSet	`postgres`	`student-rrios`	`postgres/statefulset.yaml:1-17`
Service	`postgres`	`student-rrios`	`postgres/service.yaml:1-17`
Service headless	`postgres-headless`	`student-rrios`	`postgres/service.yaml:18-35`
Job	`soloservis-migrate`	`student-rrios`	`migrations/job.yaml:1-21`
Deployment	`product-api`	`student-rrios`	`product-api/deployment.yaml:1-22`
Service	`product-api`	`student-rrios`	`product-api/service.yaml:1-17`
Deployment	`store-api`	`student-rrios`	`store-api/deployment.yaml:1-22`
Service	`store-api`	`student-rrios`	`store-api/service.yaml:1-17`
Deployment	`frontend`	`student-rrios`	`frontend/deployment.yaml:1-22`
Service	`frontend`	`student-rrios`	`frontend/service.yaml:1-17`

Tipo	Nombre	Namespace base	Archivo:líneas
ConfigMap generado	soloservis-config	student-rrios	kustomization.yaml:28-36
ConfigMap generado	soloservis-migrate-scripts	student-rrios	kustomization.yaml:37-43
ConfigMap generado	soloservis-migrations	overlay local	deploy.ps1:217-225
PVC	postgres-data-postgres-0 (lo crea el StatefulSet)	dinámico	postgres/statefulset.yaml:84-95
Secret	postgres-secret	student-rrios	postgres/secret.yaml:4-13

Todos llevan `part-of=soloservis` por la Kustomization (los ConfigMaps del overlay, por el overlay). La plantilla del PVC define `name` y `component`, pero no muestra `part-of` de forma explícita.

6.3. Migraciones en el cluster

6.3.1. De dónde salen y en qué orden se aplican

Confirmado: el README de k8s dice que el Job aplica `backend/database/migrations/*.sql` y que el overlay local se genera con esos `.sql` (`k8s/README.md:14-21`). El script lista los `.sql` del checkout actual, ordena los nombres con `Sort-Object Name`, copia cada archivo al overlay y crea un `configMapGenerator` llamado `soloservis-migrations` (`deploy.ps1:202-225`). Después llama a `kubectl kustomize` sobre el overlay (`deploy.ps1:271-278`). El Job monta ese ConfigMap en `/migrations` (`job.yaml:64-78`) y ejecuta `/scripts/migrate.sh` (`job.yaml:31-42`).

Confirmado: el migrador vuelve a ordenar los nombres completos con el `sort` del contenedor Linux y usa el nombre entero del archivo como `version` (`migrate.sh:34-43`). No extrae ni valida el prefijo numérico, así que los prefijos repetidos 006, 007 y 008 son versiones distintas, porque la clave es el nombre completo (`migrate.sh:15-18,40-44`). Si alguien agrega una migración con un prefijo que ya existe y está en el checkout, se incluye automáticamente; su posición depende del orden léxico del `sort`, no de una secuencia numérica. El script no fija `LC_ALL`, así que tampoco queda fijado el orden de mayúsculas y minúsculas entre `007_Computacion` y `007_service_cart`, ni entre `008_Telefonia` y `008_normalize_prices_to_clp`.

6.3.2. `schema_migrations`, reintentos y atomicidad

- Crea `schema_migrations` con `version TEXT PRIMARY KEY`, `checksum TEXT NOT NULL` y `applied_at TIMESTAMPTZ NOT NULL DEFAULT NOW()` (`migrate.sh:15-19`). El checksum es un SHA-256 del contenido sin CR, para tolerar CRLF y LF (`migrate.sh:42-44`).
- Si ya existe `public.user_account` pero la tabla de control está vacía, aborta y pide revisión manual (`migrate.sh:22-27`). Si una versión ya está registrada, no la repite; si el checksum cambió, emite un AVISO y igual la omite (`migrate.sh:44-52`).
- El Job es serial (`parallelism` y `completions` en 1), con `backoffLimit` 3, `activeDeadlineSeconds` 600 y `restartPolicy: Never`. No tiene `ttlSecondsAfterFinished` (`job.yaml:8-21`). El script de despliegue borra y recrea el Job en la Fase 5 (`deploy.ps1:622-629`).
- El migrador llama a `psql` con `ON_ERROR_STOP` y `-single-transaction` para cada archivo, más el INSERT del ledger (`migrate.sh:54-59`). Si falla un SQL, `psql` se detiene y se espera rollback de esa transacción; el Job reintenta según `backoffLimit`.

- **Riesgo** (estático, no ejecutado): varias migraciones de datos traen su propio `BEGIN/COMMIT` (`005_demo_catalog`, `006_demo_catalog_services`, `006_refrigeradores`, `007_Computacion`, `008_Telefonia` y `008_normalize_prices_to_clp`). Un `COMMIT` dentro de `-single-transaction` puede cerrar la transacción exterior antes de insertar en `schema_migrations`. Eso contradice el comentario "Archivo + registro en la misma transaccion" (`migrate.sh:56-58`) y podría dejar el cambio aplicado sin registro si falla el INSERT que viene después. No lo probamos.
- Idempotencia: las versiones ya registradas se omiten, pero un checksum cambiado no provoca re-ejecución. El Job, como recurso, no es idempotente por sí solo: `deploy.ps1` lo elimina y lo recrea en cada Fase 5 y se apoya en el ledger.

6.3.3. Migraciones por rama

`origin/feature/k8s-mvp-deploy` **(8):** `001_schema`, `002_views`, `003_seed`, `004_platform_apis`, `005_demo_catalog`, `006_refrigeradores`, `007_Computacion`, `008_Telefonia`.

Rama actual (11): las ocho anteriores más `006_demo_catalog_services.sql`, `007_service_cart.sql` y `008_normalize_prices_to_clp.sql`.

El script copia todos los `.sql` que haya en el checkout, así que desplegar desde la rama k8s omite esos tres cambios, y desplegar desde la actual los incluye. Lo que se puede comprobar en el SQL:

- `006_demo_catalog_services` carga catálogo y ofertas de servicios (líneas 3-799).
- `007_service_cart` extiende `cart_item` con `service_offer_id`, una restricción y un índice parcial (`007_service_cart.sql:1-12`).
- `008_normalize_prices_to_clp` crea triggers de moneda para `product_offer` y `service_offer` y normaliza las filas existentes (`008_normalize_prices_to_clp.sql:26-45`).

En la rama k8s no se aplicaría esa normalización, ni el cambio del carrito, ni esa carga demo de servicios. Las tablas base de servicios existen desde migraciones anteriores; no afirmamos que esos tres archivos las creen.

Confirmado: el README del backend menciona `008_service_cart.sql`, `009_normalize_prices_to_clp.sql` y `010_seed_all_services.sql` (`backend/README.md:1117,1122-1124`). Los nombres y números no coinciden con los archivos actuales: el `service_cart` actual es el 007, el de normalización es el 008, y `010_seed_all_services.sql` no existe en ninguna de las dos listas. La carga demo de servicios está en `006_demo_catalog_services.sql` en la rama actual. O sea que las referencias del README no calzan con el inventario que leímos, y con esto se aclara lo de 009 y 010: parecen nombres viejos o planeados, no archivos que falten.

6.3.4. ¿Se cargan seeds demo fuera de pruebas?

Sí, por diseño. El Job no distingue ambiente: monta y aplica todos los `.sql` que incluya `deploy.ps1`. En las ramas revisadas eso incluye `003_seed.sql` y `005_demo_catalog.sql`, más las cargas de catálogo 006, 007 y 008. El README declara un despliegue MVP y no condiciona el Job a un ambiente de pruebas (`k8s/README.md:14-21`; `deploy.ps1:206-225`; `migrate.sh:34-58`). No se ejecutó en un cluster ni se verificó un ambiente real.

6.4. PostgreSQL

- **StatefulSet `postgres`:** imagen `postgres:18`, una réplica, `PGDATA=/var/lib/postgresql/18/docker` y PVC `ReadWriteOnce` de 5Gi. El overlay fija la StorageClass si se elige una; si no, usa la que viene por defecto (`statefulset.yaml:10-16,37-48,84-95`; `deploy.ps1:238-245,484`).

- **Retención del PVC:** `whenDeleted: Retain` y `whenScaled: Retain` (`statefulset.yaml:14-16`).
- **Probes y recursos:** startup, readiness y liveness con `pg_isready` (líneas 64-81). Requests 100m y 256Mi, limits 250m y 512Mi (líneas 57-62).
- **Seguridad:** `runAsNonRoot` con UID y GID 999, seccomp `RuntimeDefault`, `allowPrivilegeEscalation: false` y capabilities con `drop ALL` (líneas 24-35,49-53).
- **Services:** `postgres` (ClusterIP, 5432, selector `app.kubernetes.io/name=postgres`) y `postgres-headless` (`clusterIP: None`, mismo puerto y selector) (`postgres/service.yaml:1-35`).
- **Secret:** `postgres-secret` tiene las claves `POSTGRES_USER`, `POSTGRES_PASSWORD` y `POSTGRES_DB` (`secret.yaml:4-13`). Usa `stringData`, está versionado y el propio archivo dice que son credenciales de DESARROLLO que no deben usarse en un entorno real (líneas 1-2 y 10-13). No reproducimos los valores.
- **No encontré:** backups, réplicas o alta disponibilidad de PostgreSQL, NetworkPolicy y PodDisruptionBudget en los 14 archivos de `k8s/base` (el StatefulSet tiene una sola réplica).

La Fase 1 del script y el PVC. Borra Deployment, StatefulSet, Service, ConfigMap, Secret, Job, Ingress y Pod con la etiqueta `part-of=soloservis`, más algunos nombres heredados concretos. No borra el PVC por selector: `postgres-data-postgres-0` solo se elimina si su fase no es `Bound`; si está `Bound`, se conserva (`deploy.ps1:341-365`). El riesgo es que el selector actúa en el namespace elegido, y el script no compara el cluster con una identidad aprobada: la Fase 0 solo prueba `kubectl auth can-i list pods` y muestra el inventario (`deploy.ps1:288-327`). No hay evidencia de que la Fase 1 borre un PVC de Postgres `Bound`, pero sí puede borrar un claim que no lo esté y los recursos con esa etiqueta en un namespace equivocado o compartido. Qué pasa con el volumen físico depende de la StorageClass y de su reclaim policy, que no están versionadas: **No encontré:** eso en los archivos.

Reinicios y datos. Reiniciar `postgres-0` conserva el PVC y está pensado para mantener los datos. La Fase 10 compara el conteo del ledger y el UID y estado del PVC antes y después, pero esa prueba no se ejecutó en esta auditoría (`deploy.ps1:797-805`). Si alguien borra el PVC a mano, se puede perder la persistencia; si el volumen físico se conserva o no depende de una política externa que los archivos no indican. No hay backup descrito.

Usuarios y permisos. `product-api`, `store-api` y el Job toman `POSTGRES_USER`, `POSTGRES_PASSWORD` y `POSTGRES_DB` del mismo Secret y se conectan al Service `postgres` (`deployment.yaml` de las APIs:44-57; `job.yaml:34-50`). No hay roles separados ni GRANT de mínimo privilegio en estos manifests. Los permisos reales no se pueden verificar sin consultar la base, y no lo hicimos.

6.5. APIs, frontend y Dockerfiles

Deployment	Imagen en el manifest	Tag	Puerto	Escala, seguridad y health
`product-api`	`soloservis/product-api`	base `dev`; el overlay cambia registry y tag	8081	1 réplica, UID 10001, `readOnlyRootFilesystem`, resources; readiness `/health` y liveness TCP
`store-api`	`soloservis/store-api`	base `dev`; el overlay cambia registry y tag	8082	igual; readiness `/health` y liveness TCP

Deployment	Imagen en el manifest	Tag	Puerto	Escala, seguridad y health
frontend	soloservis/frontend	base dev; el overlay cambia registry y tag	8080	1 réplica, UID 101, readOnlyRootFilesystem, resources; probes en /healthz

Evidencia: product-api/deployment.yaml:9,34-42,44-82; store-api/deployment.yaml:9,34-42,44-82; frontend/deployment.yaml:9,29-53. Los Services de las APIs son ClusterIP en 8081 y 8082; el del frontend es ClusterIP en el puerto 80 hacia el targetPort nombrado http (frontend/service.yaml:1-17). El Ingress apunta al puerto nombrado http del frontend, que es el 80 del Service y luego el 8080 del contenedor (ingress.yaml:18-25).

- Los tres Deployments usan imagePullPolicy: IfNotPresent. El overlay elige newName y newTag; el tag por defecto del script es un SHA corto, y si Git está sucio le suma un timestamp (deploy.ps1:191-200,229-236,843-850). No usa latest por defecto, pero se puede pasar cualquier Tag; un tag mutable junto con IfNotPresent puede dejar una imagen local vieja. No hay digest fijado.
- DATABASE_URL se arma en cada API con el Service postgres, el nombre de la base y variables secretas; en la configuración aparece sslmode=disable (deployment.yaml de ambas APIs:44-57). FRONTEND_URL viene de un ConfigMap (APIs:59-61; kustomization.yaml:30-36; overlay:229-236).
- **Dockerfile de las APIs:** multistage con builder golang:1.27-alpine; compila ./cmd/product-api o ./cmd/store-api, el runtime es alpine:3.22, copia solo el binario y corre como 10001:10001, con los puertos 8081 y 8082 (líneas 3-26 de cada uno). El go.mod declara go 1.27.1 (go.mod:3), y el tag del builder no fija ese patch.
- **Dockerfile del frontend:** multistage de node:24-alpine a nginxinc/nginx-unprivileged:1.29-alpin los argumentos VITE_*_API_URL vienen vacíos por defecto y se copia dist a nginx (líneas 4-44). Dockerfile.dockerignore excluye .env y .env.* del frontend (líneas 1-10) y backend/.dockerignore hace lo mismo con el backend (líneas 1-8). Aunque COPY . . mete las fuentes en la etapa builder, las etapas finales copian solo el binario o dist, no el contexto completo.
- El script solo construye esas tres imágenes. cmd/product-api llama a NewProductRouter (backend/cmd/product-api/main.go:30) y cmd/store-api a NewStoreRouter (backend/cmd/store-api cmd/server llama a NewRouter (backend/cmd/server/main.go:32), pero el script no lo construye (deploy.ps1:557-561).
- Los puertos coinciden en todas partes: el README de k8s dice nginx 8080, product-api 8081 y store-api 8082 (k8s/README.md:4-7); los Dockerfiles hacen EXPOSE 8080, 8081 y 8082; los containerPort y los Services calzan; y los entrypoints de Go leen PORT y sirven sus routers por separado (cmd/*/main.go:30-37; deployments:40-42).
- En seguridad de los contenedores: una réplica de cada uno, requests y limits, runAsNonRoot, seccomp RuntimeDefault, allowPrivilegeEscalation: false, drop ALL y readOnlyRootFilesystem en APIs y frontend. No hay imagePullSecrets por defecto; el overlay lo agrega solo si se pasa -ImagePullSecret (deploy.ps1:250-257).

6.6. Ingress y nginx

Método	Ruta externa	Destino / handler	Autenticación o restricción
GET	`/products` y subrutas	nginx → `product-api:8081`; Go registra `GET /products` y `GET /products/{publicID}`	El router solo instala CORS, sin auth; el proxy deja pasar GET
POST	`/products`	`product-api`, handler `Create`	Sin auth en el router, pero nginx tiene `limit_except GET { deny all; }` y bloquea todo lo que no sea GET
PUT	`/products/{publicID}`	`product-api`, handler `Update`	Igual: bloqueado en el proxy
DELETE	`/products/{publicID}`	`product-api`, handler `Deactivate`	Igual: bloqueado en el proxy
GET	`/stores` y `/stores/{id}`	nginx → `store-api:8082`	El router solo tiene CORS y rutas GET; el proxy deja pasar GET
cualquiera	`/`	Ingress → Service `frontend`:80 → nginx:8080	Fallback de la SPA para lo que no es proxy
GET/POST	`/offers`, `/ingestion/*`, `/search/*`	Sin location de proxy; en el código existen en los routers generales, pero sus Deployments están fuera del MVP	No quedan expuestas por el Ingress descrito; una ruta desconocida cae en la SPA

Evidencia: el API de producto está en `backend/internal/http/router.go:42-56` (`NewProductRouter`, CORS y GET/POST/PUT/DELETE, sin middleware de auth); el de tiendas en `router.go:77-90` (CORS y solo GET); pricing e ingesta en `backend/internal/features/router.go:11-22,96-98`, que no son servicios incluidos en la base. Nginx: `docker/frontend/nginx.conf:18-29` (`/products`), `31-42` (`/stores`) y `45-54` (assets y SPA). Ingress: `k8s/base/ingress.yaml:8-25`.

- El Ingress usa la clase `nginx` y la anotación `external-dns.alpha.kubernetes.io/target`, con solo el path `/` hacia el Service `frontend`. El host lo agrega el overlay. No hay sección `tls:` en `ingress.yaml` (líneas 8-25). El script acepta esquema http o https y usa http por defecto (`deploy.ps1:40-41`); el manifiesto no configura TLS ni certificado. El host por defecto es `soloservis-<usuario>.dev.censei.cl` (`deploy.ps1:839-852`).
- nginx escucha en el 8080. No declara `client_max_body_size` ni `limit_req`. Sí fija `Host`, `X-Forwarded-For` y `X-Forwarded-Proto`, y timeouts de conexión y lectura (líneas 18-42).
- Por el `limit_except GET`, nginx impide POST, PUT y DELETE hacia `/products` y `/stores` por el host público: a través de ese proxy no hay permiso de escritura público. Esto matiza el hallazgo del Paso 3: las rutas de escritura sin autenticación siguen existiendo en el API, y quedarían accesibles para quien pueda llegar directo al Service o al API desde dentro de la red. Los Services son ClusterIP y no hay NetworkPolicy, así que no es una exposición pública directa, pero tampoco hay aislamiento de red expresado en estos manifests.
- `/offers`, `/ingestion` y `/search` no tienen proxy y esos servicios están fuera del MVP (README:54-56); en `location /` el navegador recibe el fallback de la SPA, no esas APIs (`nginx.conf:45-54`).

- Mismo origen: el README dice que el frontend se compila con `VITE_*_API_URL=""` (`k8s/README.md:16-2` y el ARG del Dockerfile tiene el default vacío (`Dockerfile` del frontend:19-34). El navegador usa el host del Ingress y nginx hace el proxy a los Services. El backend mantiene CORS configurado desde `FRONTEND_URL`, pero el flujo descrito no depende de CORS del navegador para esos endpoints.

6.7. El script `scripts/k8s/deploy.ps1`

Parámetros y defaults (`deploy.ps1:27-46,51-82,839-857`): `Kubeconfig` vacío usa `KUBECONFIG`, `~/.kube/config` o el único `Downloads/estudiantes-*.kubeconfig`; `Context` y `Namespace` vacíos toman los del kubeconfig; `Registry`, `Tag`, `StorageClass`, `IngressClass`, `IngressHost` e `ImagePullSecret` vienen vacíos; `IngressScheme` es `http`; `FromPhase=0`, `ToPhase=10`; `SkipCleanup` apagado. El Tag por defecto es un SHA (con timestamp si Git está sucio) y el host por defecto es `soloservis-<usuario>.dev.censei.cl`.

Fase	Qué hace el script	Efecto si se ejecuta
0	Comprueba repo, CLI y kubeconfig, pide `auth can-i list pods`, lista recursos, Secrets solo por nombre, quota, clases e inventario (líneas 288-339)	Lee del cluster y se autentica; no aplica recursos
1	Limpieza por etiqueta y por nombres heredados; el PVC de Postgres solo si no está `Bound` (341-365)	Elimina recursos de Kubernetes
2	Prueba la StorageClass con un PVC y un Pod temporales (escribe un archivo y prueba persistencia y DNS), elige la IngressClass y genera y renderiza el overlay (367-554)	Crea y borra recursos de prueba; escribe el overlay local ignorado por git
3	`docker build`, `push` y `manifest inspect` de las tres imágenes (555-585)	Construye y publica imágenes en el registry
4	Aplica Secret, Services y StatefulSet de PostgreSQL y espera el rollout y el PVC (590-620)	Modifica el cluster
5	Borra y recrea el Job de migración, lo aplica, espera, lee logs y consulta `schema_migrations` y `product` (622-666)	Modifica el cluster y la base, a través del Job
6	Despliega `product-api` (702)	Aplica Deployment y ConfigMap y hace health check
7	Despliega `store-api` (703)	Aplica Deployment y ConfigMap y hace health check
8	Despliega `frontend` (704)	Aplica el Deployment y hace health check
9	Aplica el Ingress y hace pruebas HTTP externas a `/`, `/products` y `/stores` (733-771)	Modifica el cluster y llama al host externo
10	Inspecciona recursos, prueba comunicación, elimina el pod de `product-api` y el `postgres-0` y espera que vuelvan (773-831)	Elimina pods para probar recreación y persistencia; lee la base

- Hay un checkpoint por fase (función `Checkpoint`, `deploy.ps1:100-119`). No se ejecutó el script.
- La Fase 0 no valida una identidad de cluster aprobada: solo mira que existan el kubeconfig y el namespace, y que `can-i list pods` responda bien. La Fase 1 tiene alcance de namespace

más selector, así que un kubeconfig o namespace equivocado podría apuntar a recursos ajenos con la misma etiqueta.

- El manejo de errores usa helpers de salida y códigos, con `Stop-Phase`. Conviene revisar los checkpoints antes de ejecutar, sobre todo en las fases 1, 5 y 10.
- No imprime valores de Secret: la Fase 0 solo lista nombres.
- **Generación del overlay:** crea `k8s/overlays/local`, borra los SQL anteriores dentro de ese overlay, copia los `*.sql` del checkout actual y arma una Kustomization con namespace, `configMapGenerator`, imágenes, StorageClass, IngressClass y host, e `ImagePullSecret` opcional (`deploy.ps1:202-269`). `kubectl kustomize` lo invoca `Build-Rendered` cuando se ejecuta el script (líneas 271-278), pero no lo corrimos. Esa generación sí puede dejar fuera migraciones, porque solo copia lo que haya en el checkout.
- **Ejecución repetida:** la limpieza vuelve a borrar los recursos etiquetados y conserva el PVC solo si está `Bound`. La fase de migraciones recrea el Job, pero el ledger omite las versiones ya aplicadas; si un SQL cambió después de aplicarse, solo avisa y lo omite. Con Git limpio el tag se repite por SHA; con Git sucio se genera un timestamp nuevo. No hay garantía de que `IfNotPresent` descargue una imagen reemplazada con el mismo tag.

6.8. Seguridad y operación

- **Alto:** el Secret de desarrollo está como `stringData` en un YAML versionado, con las tres claves `POSTGRES_*`. El propio manifiesto prohíbe usarlo en un entorno real y pide crear el Secret fuera de git (`secret.yaml:1-13`). No se imprimieron credenciales.
- **Alto:** la Fase 1 es destructiva y confía en el contexto y namespace del kubeconfig; el selector `part-of` y los nombres heredados pueden afectar recursos del namespace (`deploy.ps1:341-365`). Hay que validar a mano el destino antes de cualquier uso. No se ejecutó.
- **Medio:** el runner usa `-single-transaction`, pero un SQL con `COMMIT` propio puede terminar la transacción exterior antes de registrar el checksum (`migrate.sh:56-58`). La garantía atómica no está probada y podría ser falsa para esos archivos.
- **Medio:** una sola réplica de Postgres, sin backup ni HA en los manifests. Perder el PVC puede significar perder la base. `Retain` protege el PVC ligado al StatefulSet y la Fase 1 preserva el `Bound`; la política del volumen de fondo no está en el repositorio.
- **Medio:** no hay TLS declarado en el Ingress y el esquema por defecto del script es http (`ingress.yaml:8-25`; `deploy.ps1:40-41`). La plataforma podría terminar TLS por fuera, pero **No encontré:** eso y no lo inferimos.
- **Medio:** no se encontraron NetworkPolicy, PodDisruptionBudget ni rate limiting en nginx. Los Services ClusterIP no son endpoints públicos por sí solos.
- **Bajo (operativo):** los tags no fijan digest; Postgres usa `postgres:18`, el builder de las APIs `golang:1.27-alpine` y la política es `IfNotPresent`. El script genera un tag por SHA, pero acepta cualquier Tag y el overlay lo aplica.
- **Lo bueno:** PostgreSQL, APIs y frontend corren sin root y con capabilities limitadas; APIs y frontend tienen sistema de archivos raíz de solo lectura; todos los Deployments tienen resources y probes; Postgres tiene tres probes y resources (`statefulset.yaml:24-81`). El Job también corre sin root, con root de solo lectura, resources y `restartPolicy: Never` (`job.yaml:21-60`).
- **Permisos en PostgreSQL:** las APIs y el Job comparten las claves y el mismo usuario; no hay roles separados ni GRANT expresados. Los permisos concretos no se pueden verificar en modo estático.

6.9. Pruebas y validación

- `.github/workflows` no existe en el checkout. **No encontré:** un workflow de CI/CD de Kubernetes, `kubeconform` o pruebas automatizadas de los manifests.
- Tampoco hay un render o prueba de manifests en CI. `deploy.ps1` contiene `kubectl kustomize`, pero solo lo llama durante la ejecución del despliegue (`deploy.ps1:271-278`).
- `kubectl kustomize` sobre la base y el overlay: **no ejecutado**. Requería confirmación según las reglas del encargo y, además, el overlay local no existe ni se generó. No se aplicó nada.

6.10. Contraste con los Pasos 1 a 5

Hallazgo previo	Qué muestra Kubernetes	Resultado
Línea A de 11 migraciones	La rama k8s tiene 8, la actual tiene 11, y el script toma lo que haya en el checkout	Difiere según la rama; riesgo de desplegar un esquema distinto
Prefijos 006/007/008 repetidos	Cada nombre completo es una clave distinta y el orden lo da el `sort` del contenedor, con locale sin fijar	Compatible, pero el orden exacto no queda fijado
Orden A/B probado en el Paso 5	Según lo ya reportado, ambos órdenes terminaron sin error en las bases temporales; no se repitió aquí	Coincide para esa prueba demo; no prueba nada del cluster
Migraciones 009/010 del README	El README del backend habla de 008, 009 y 010; los archivos reales son 007 y 008, y la carga de servicios está en el 006	Difiere la documentación del inventario
Trigger CLP	`008_normalize_prices_to_clp` existe solo en la rama actual; la rama k8s no lo monta	Difiere; desplegando la rama k8s esa migración no fuerza ni normaliza CLP
Rutas sin autenticación (Paso 3)	El API de producto tiene escrituras sin middleware de auth; nginx solo hace proxy de GET	Las escrituras quedan bloqueadas en el acceso público; no hay proxy para ofertas, ingesta ni búsqueda
Ausencia de pruebas (Paso 4)	No hay workflow, `kubeconform` ni pruebas de manifests	Coincide; no se ejecutaron pruebas de k8s
Versión de Go	`go.mod` dice 1.27.1; el builder es `golang:1.27-alpine`	Difiere en la fijación exacta del patch
Ingestion y pricing	El README de k8s los deja fuera del MVP y el script construye solo product, store y frontend	Coincide; este manifiesto no los despliega

Sobre el orden dentro de los pares. `006_demo_catalog_services` y `006_refrigeradores` car-

gan catálogos distintos. `007_Computacion` y `007_service_cart` operan en áreas distintas (`cart_item` existe desde 004). El código que vimos no establece dependencia entre los miembros de cada par. En el 008, si la normalización corre antes de Telefonia, fuerza CLP en cada insert o update; si corre después, recorre y normaliza las ofertas que ya existen (`008_normalize_prices_to_clp.sql:26-45`; `008_Telefonia.sql:212-248`). Por eso, según esas sentencias, la moneda final converge a CLP en ambos órdenes y solo cambia el momento de la normalización. El Paso 5 ya reportó ambos órdenes sin error en bases descartables, aunque no repetimos esa comparación aquí. El orden automático real depende del `sort` y del locale, que el script no fija.

6.11. Hallazgos nuevos y preguntas

Hallazgos prioritarios:

1. **Alto:** la Fase 1 puede borrar recursos etiquetados en el namespace configurado sin comprobar la identidad esperada del cluster; hacen falta guardas o revisión manual antes de ejecutarla (`deploy.ps1:341-365,288-327`).
2. **Alto:** hay credenciales de desarrollo como `stringData` en un Secret versionado (`secret.yaml:1-13`); hay que reemplazarlas por un Secret gestionado fuera del repositorio antes de cualquier uso real.
3. **Medio:** posible pérdida de atomicidad entre migración y ledger por el `COMMIT` interno combinado con `-single-transaction` (`migrate.sh:56-58` y las migraciones con `BEGIN/COMMIT`). Conviene corregirlo o verificarlo antes de confiar en los reintentos.
4. **Medio:** desplegar desde la rama k8s deja fuera las tres migraciones de la línea actual (catálogo de servicios, `service_cart` y normalización CLP).
5. **Medio:** no hay TLS declarado, ni backups, ni HA, ni NetworkPolicy, ni PDB en los manifests revisados.

Preguntas abiertas para el equipo:

- ¿Qué rama o commit es la fuente oficial para desplegar? ¿Se debe exigir el conjunto de 11 migraciones?
- ¿Se actualiza `backend/README.md` para que los nombres y números reales (006, 007, 008) coincidan con las referencias 008, 009 y 010?
- ¿Se eliminan los `BEGIN/COMMIT` internos o se ajusta el runner para garantizar la atomicidad archivo más ledger?
- ¿Cómo se entregará el Secret real y qué rol mínimo debe usar cada API y el Job?
- ¿Dónde termina TLS y cuál es la política externa de retención, reclaim y backup del PVC?
- ¿La Fase 1 debería exigir confirmación, identidad de cluster y un namespace exacto antes de borrar?

6.12. Cierre

Render: **no ejecutado**. Cluster: **no consultado**. Docker: **no ejecutado**. SQL: **no ejecutado** en este paso. No se corrió `deploy.ps1` ni se usó ningún kubeconfig. No se tocó el cluster, Docker, SQL ni ninguna base; no se usó `docker compose` ni se leyó su contenido; no se modificó ningún archivo del repositorio. `git status -short` y `git diff -stat`: sin salida. El único archivo creado para este entregable está en la carpeta TEMP indicada en la ruta del informe.